Proyecto Final — Análisis de Algoritmos

Análisis de algoritmos en el contexto de la bibliometría

Similitud textual, agrupamiento jerárquico y búsqueda de patrones sobre un corpus de 20 artículos científicos

Mariana Muñoz Rodriguez
Andres Felipe Correa Valencia
Ivan Alexander Cardona
Milton Cesar Gomez Roso



Profesor: Sergio Augusto Cardona Torres

Universidad del Quindío
Facultad de Ingeniería
Ingeniería de Sistemas y Computación
Análisis de Algoritmos
Armenia, Quindío, 2026

Índice

1. Introducción **1**

2. Arquitectura del sistema y herramientas **2**
2.1. Herramientas y tecnologías . . . 2
2.2. Arquitectura por capas . . . 2
2.3. Estructura del proyecto . . . 4
2.4. Flujo de una solicitud . . . 4

3. Corpus y extracción de metadatos **4**
3.1. Fuente de información . . . 4
3.2. Algoritmo de extracción . . . 5
3.3. Validación . . . 6

4. Preprocesamiento del texto **6**

5. Requerimiento 1: Análisis de similitud textual **7**
5.1. Flujo general del comparador . . . 7
5.2. Caso de estudio . . . 8
5.3. Algoritmo clásico 1: distancia de Levenshtein . . . 8
5.4. Algoritmo clásico 2: alineamiento global de Needleman-Wunsch . . . 10
5.5. Algoritmo clásico 3: coeficiente de Jaccard . . . 12
5.6. Algoritmo clásico 4: similitud del coseno sobre TF-IDF . . . 13
5.7. Modelo de IA 1: GloVe con promedio de embeddings . . . 15
5.8. Modelo de IA 2: Sentence-BERT . . . 16
5.9. Análisis crítico: enfoques clásicos frente a modelos de IA . . . 17

6. Requerimiento 2: Agrupamiento jerárquico **20**
6.1. Flujo general . . . 20
6.2. Algoritmo aglomerativo genérico . . . 20
6.3. Criterios de enlace . . . 21
6.4. Ejemplo paso a paso . . . 22
6.5. Métricas de evaluación interna . . . 23
6.6. Resultados . . . 24

7. Requerimiento 3: Búsqueda de patrones **29**
7.1. Flujo general . . . 29
7.2. Caso de estudio . . . 29
7.3. Algoritmo de Knuth-Morris-Pratt . . . 30
7.4. Algoritmo de Boyer-Moore . . . 33
7.5. Algoritmo de Rabin-Karp . . . 36

7.6. Análisis de complejidad y contraste empírico 40

8. Requerimiento 4: Despliegue y documentación técnica 42

8.1. Ejecución local por consola 42
8.2. API REST 43
8.3. Interfaz web 43
8.4. Despliegue con Docker en OVHcloud 50
8.5. Pruebas y validación 50
8.6. Documentación técnica 51

9. Declaración de uso de IA generativa 51

9.1. IA como componente de la solución 51
9.2. IA generativa como herramienta de apoyo al desarrollo 52

10. Conclusiones 52

Referencias 53

A. Corpus completo 54

B. Reproducción de los resultados 56

C. Primeras fusiones del agrupamiento con average linkage 56

Índice de figuras

1. Arquitectura por capas: fuente, extracción, preprocesamiento, algoritmos (R1–R3) y presentación (R4). La interfaz se comunica con la API mediante HTTP/JSON. 3
2. Longitud (en palabras) del abstract de cada artículo. 5
3. Primeras 22 columnas del alineamiento global óptimo de los abstracts completos («|» coincidencia, «·» discrepancia, «–» hueco). 11
4. Similitud entre A11 y A16 según los seis algoritmos. 17
5. Matrices de similitud 20 × 20 de cada algoritmo; la escala de cada panel va del mínimo al máximo fuera de la diagonal. 18
6. Distribución de las 190 similitudes por algoritmo; el punto marca el par del caso de estudio. 19
7. Coeficiente de silueta según el número de grupos k. 25
8. Correlación cofenética por criterio y representación. 25
9. Dendrograma *single linkage* (TF-IDF, distancia coseno). 26
10. Dendrograma *complete linkage* (TF-IDF, distancia coseno). 26
11. Dendrograma *average linkage* (TF-IDF, distancia coseno). 27
12. Dendrograma de Ward (TF-IDF, distancia euclidiana), criterio adicional. 27
13. Dendrograma *complete linkage* sobre los embeddings de Sentence-BERT. 28
14. Trayectoria de las comparaciones de cada algoritmo sobre el párrafo: posición del texto comparada (eje x) en cada paso (eje y, hacia abajo). Las franjas amarillas son las ocurrencias. 30
15. Resultados empíricos: tiempo de búsqueda y comparaciones según m, tiempo según n y tiempo de preprocesamiento según m. 40
16. Interfaz web: página del corpus. 44
17. Interfaz web: comparación de cuatro artículos con los seis algoritmos (R1). 45
18. Interfaz web: demostración paso a paso del caso de estudio (R1). 46
19. Interfaz web: dendrograma interactivo y evaluación interna (R2). 47
20. Interfaz web: búsqueda de «generative artificial intelligence» en abstracts y títulos (R3). 48
21. Interfaz web: reproductor paso a paso de Boyer-Moore con sus tablas heurísticas (R3). 49
22. Documentación interactiva de la API (OpenAPI) en `/docs`. 50

Índice de tablas

1. Tecnologías utilizadas y su función en el proyecto. 2
2. Organización de carpetas y archivos principales. 4
3. Algoritmos del Requerimiento 1. 7
4. Matriz D de Levenshtein sobre las primeras 8 raíces de A11 (filas) y A16 (columnas). . 9
5. Matriz F de Needleman-Wunsch (coincidencia $+1$, discrepancia -1, hueco -1). 11
6. Pesos TF-IDF de los términos compartidos por A11 y A16 ($\hat{w}$: pesos normalizados con la norma L2). 14

7. Vectores GloVe de algunas palabras de A11 y vectores promedio de ambos documentos (primeras 6 de 100 dimensiones). . . . 15
8. Embeddings Sentence-BERT de A11 y A16 (primeras 8 de 384 dimensiones; vectores con norma 1). . . . 16
9. Estadísticas de cada algoritmo sobre los 190 pares (media, desviación estándar, rango, par más similar, valor y posición del caso de estudio, y tiempo total de cálculo). . . . 18
10. Correlación de rangos de Spearman entre algoritmos sobre los 190 pares. . . . 19
11. Criterios de enlace: definición y coeficientes de Lance-Williams ($N = n_h + n_i + n_j$). . . . 21
12. Distancias coseno entre cinco abstracts (TF-IDF). . . . 22
13. Iteraciones con *single linkage*. . . . 22
14. Iteraciones con *complete linkage*. . . . 22
15. Iteraciones con *average linkage*. . . . 22
16. CPCC y silueta para $k = 2, \ldots, 8$. En negrilla, el criterio más coherente entre los tres exigidos. . . . 24
17. Resultados del caso de estudio ($n = 267$, $m = 10$). Rabin-Karp con $q = 13$. . . . 29
18. Función de prefijos de `"assessment"`. . . . 31
19. Función de prefijos de `"interdisciplinary"`. . . . 31
20. Fragmento de la traza de KMP (i: posición en el texto, q: caracteres emparejados, $s = i - q$: alineación). . . . 32
21. Tabla del carácter malo de `"assessment"`. . . . 33
22. Tabla del sufijo bueno gs de `"assessment"` (falla en $P[j] \Rightarrow$ se usa $gs[j+1]$). . . . 34
23. Iteraciones de Boyer-Moore sobre el texto del caso de estudio. . . . 34
24. Hash del patrón por la regla de Horner ($d = 256$, $q = 13$). . . . 37
25. Rolling hash en las primeras ventanas del texto. . . . 37
26. Ventanas candidatas ($t_s = H(P) = 4$) con $q = 13$. . . . 38
27. Complejidad asintótica teórica. . . . 40
28. Ajuste lineal del tiempo de búsqueda frente a n (experimento B): costo por carácter y coeficiente R^2. . . . 41
29. Comparaciones en los peores casos teóricos ($T = a^n$, $n = 20\,000$). . . . 41
30. Colisiones de Rabin-Karp según el módulo q (patrón `learning` sobre los 20 abstracts, $n \approx 28\,000$). . . . 41
31. Búsqueda de `"assessment"` en los 20 abstracts (28 244 caracteres; 37 ocurrencias en 11 artículos). . . . 42
32. Opciones del menú por consola. . . . 43
33. Endpoints de la API. . . . 43
34. Estrategia de pruebas (177 passed in 10.36s con `pytest`; 10 pruebas de extremo a extremo con Playwright). . . . 51
35. Los 20 artículos del corpus con los metadatos extraídos automáticamente. . . . 54
36. Primeras 8 fusiones de *average linkage* sobre el corpus completo (TF-IDF). C_t es el clúster creado en el paso t. . . . 56

1. Introducción

El estudio computacional de la producción científica permite describir, relacionar y presentar grandes volúmenes de texto con métodos cuantitativos. En ese contexto, tres tareas algorítmicas son fundamentales: *medir la similitud* entre documentos, *agruparlos* según esa similitud y *localizar patrones* exactos dentro de ellos. Este proyecto aborda las tres sobre un corpus controlado de 20 artículos científicos en PDF, entregados por el docente, cuyo dominio es la inteligencia artificial (IA) en la educación y, en particular, la IA generativa.

El procesamiento se centra en el *abstract* de cada artículo; el título y los autores se extraen además para cruzar información (por ejemplo, para etiquetar los dendrogramas o para buscar un autor en todo el corpus). El objetivo no es sólo obtener resultados, sino hacer visible el funcionamiento interno de cada algoritmo y analizarlo con rigor: llenado de matrices de programación dinámica, cálculo de pesos y ángulos en el modelo vectorial, actualización de distancias entre clústeres, funciones de prefijos, tablas heurísticas y *hashes* rodantes.

Para cumplirlo se construyó una aplicación modular en Python con estas características:

- **Extracción automática de metadatos** (título, autores, abstract, palabras clave, año, revista y DOI) a partir de la geometría de la primera página de cada PDF, validada campo por campo contra una referencia verificada manualmente (140/140 campos correctos).
- **Requerimiento 1 — Similitud textual**: cuatro algoritmos clásicos (Levenshtein y Needleman-Wunsch, de programación dinámica; Jaccard, de conjuntos; coseno sobre TF-IDF, de espacio vectorial) y dos modelos de IA basados en *embeddings* (GloVe y Sentence-BERT).
- **Requerimiento 2 — Agrupamiento jerárquico**: algoritmo aglomerativo genérico con la fórmula de Lance-Williams y los criterios *single*, *complete* y *average linkage* (más Ward como complemento), dendrogramas y evaluación interna con la correlación cofenética y el coeficiente de silueta.
- **Requerimiento 3 — Búsqueda de patrones**: Knuth-Morris-Pratt, Boyer-Moore y Rabin-Karp, con trazas paso a paso y un estudio empírico que contrasta la complejidad teórica con el rendimiento observado.
- **Requerimiento 4 — Despliegue**: API REST, interfaz web interactiva, menú por consola, imagen Docker para el despliegue en un servidor de OVHcloud con dominio propio, y documentación técnica.

Siguiendo la exigencia del curso, **ningún algoritmo solicitado se delega en funciones de alto nivel**: todos están escritos con listas, diccionarios y ciclos explícitos. Las librerías científicas (SciPy, scikit-learn, NLTK y RapidFuzz) se emplean *únicamente en las pruebas automatizadas*, como oráculos que confirman que las implementaciones propias producen exactamente los mismos resultados.

El documento se organiza así: la sección 2 presenta la arquitectura y las herramientas; la 3 describe el corpus y la extracción de metadatos; la 4, el preprocesamiento común; las secciones 5, 6 y 7 desarrollan los tres requerimientos funcionales con sus casos de estudio; la 8 documenta el despliegue y las pruebas, y las últimas secciones declaran el uso de IA generativa y presentan las conclusiones.

2. Arquitectura del sistema y herramientas

2.1. Herramientas y tecnologías

Tabla 1: Tecnologías utilizadas y su función en el proyecto.

Herramienta	Uso en el proyecto
Python 3.11	Lenguaje del núcleo algorítmico, la API y la consola.
PyMuPDF [29]	Lectura de las líneas de texto de los PDF con su posición y tamaño de fuente (la lógica de reconocimiento es propia).
FastAPI + Uvicorn [28]	API REST que expone los tres requerimientos y sirve la interfaz compilada; documentación OpenAPI automática.
GloVe (Stanford)	Embeddings preentrenados de palabras (modelo de IA 1).
Sentence-Transformers	Generación de los embeddings del modelo `all-MiniLM-L6-v2` (modelo de IA 2).
React, TypeScript, Vite	Interfaz web de una sola página.
shadcn/ui [31] + Tailwind CSS	Componentes de interfaz accesibles y sistema de estilos.
Recharts [32], KaTeX, SVG	Gráficas interactivas, fórmulas matemáticas y dendrogramas dibujados con código propio.
Matplotlib	Figuras del informe y dendrogramas PNG generados desde la consola.
pytest, Hypothesis, Playwright	Pruebas unitarias, de propiedades, de integración, de flujo y de extremo a extremo.
SciPy [33], scikit-learn [34], NLTK, RapidFuzz	**Sólo en pruebas**, como oráculos de referencia.
Docker, Docker Compose, Caddy	Empaquetado y despliegue en un servidor de OVHcloud con dominio propio y HTTPS.

2.2. Arquitectura por capas

La figura 1 resume el diseño. La aplicación se organiza en capas con dependencias en un solo sentido: los algoritmos no conocen la API ni la interfaz, y pueden usarse desde la consola, desde las pruebas o desde el servicio web.

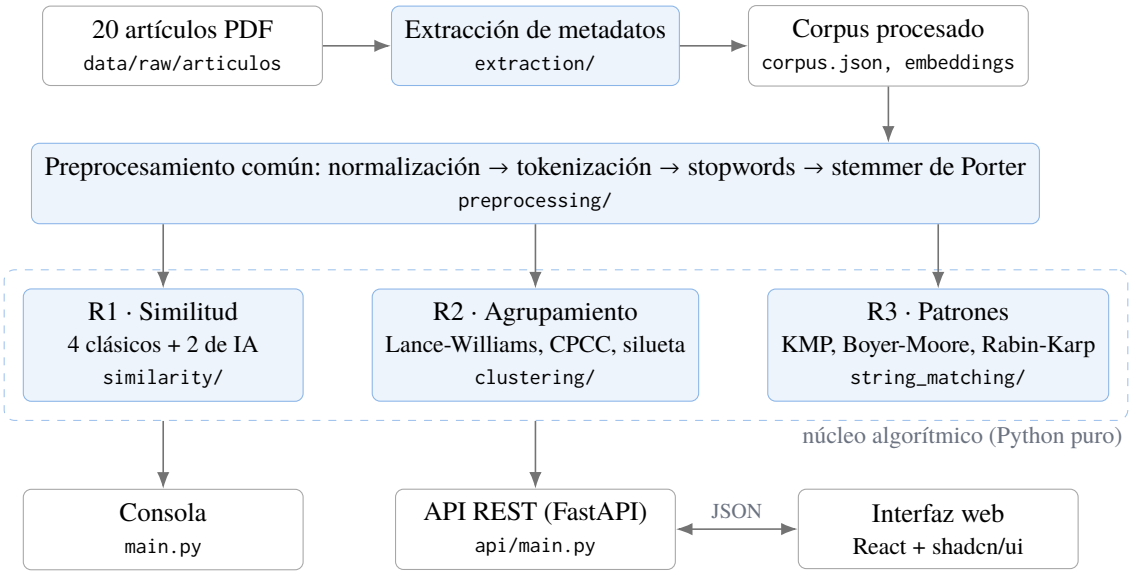


Figura 1: Arquitectura por capas: fuente, extracción, preprocesamiento, algoritmos (R1–R3) y presentación (R4). La interfaz se comunica con la API mediante HTTP/JSON.

Decisiones de diseño.

- **Algoritmos puros y transparentes.** Cada algoritmo vive en su propio módulo, recibe estructuras básicas (cadenas, listas, diccionarios) y devuelve un resultado con sus estructuras auxiliares (matrices, tablas, trazas), de modo que la misma función alimenta la consola, la interfaz y las demostraciones de este informe.
- **Representación común.** Los cuatro algoritmos clásicos reciben la misma secuencia de raíces; así las diferencias en los resultados se deben al algoritmo y no al preprocesamiento.
- **Embeddings precalculados.** Como el corpus es estático, los vectores de los modelos de IA se generan una sola vez (`scripts/build_embeddings.py`) y se guardan en `data/`. La aplicación desplegada no necesita PyTorch: la selección de artículos sigue siendo dinámica y la similitud (coseno y distancia euclidiana) se calcula en línea con código propio. Es el mismo principio de un índice vectorial.
- **Trazas acotadas.** Los algoritmos de búsqueda pueden registrar cada comparación y desplazamiento (con un tope de eventos) para reproducirlos paso a paso en la interfaz; sin traza se ejecutan sin sobrecosto para medir tiempos.
- **Un solo despliegue.** FastAPI sirve la API (`/api/*`) y los archivos estáticos de la interfaz, por lo que una única imagen Docker contiene todo el sistema.

2.3. Estructura del proyecto

Tabla 2: Organización de carpetas y archivos principales.

Ruta	Contenido
`bibliometria/extraction/`	`pdf_extractor.py` (reconocimiento de campos) y `build_corpus.py` (genera `corpus.json`).
`bibliometria/preprocessing/`	Tokenizador, lista de stopwords y stemmer de Porter propio.
`bibliometria/similarity/`	R1: `levenshtein.py`, `needleman_wunsch.py`, `jaccard.py`, `tfidf.py`, `embeddings.py`, `vector_math.py` y `service.py`.
`bibliometria/clustering/`	R2: `agglomerative.py`, `evaluation.py`, `dendrogram.py` y `service.py`.
`bibliometria/string_matching/`	R3: `kmp.py`, `boyer_moore.py`, `rabin_karp.py`, `naive.py` (referencia), `benchmark.py` y `service.py`.
`bibliometria/api/main.py`	R4: API REST y servidor de la interfaz.
`data/raw/articulos/`	Los 20 PDF entregados por el docente.
`data/processed/`	`corpus.json`, `embeddings_sbert.json`, resultados del benchmark.
`data/models/`	Subconjunto de GloVe con el vocabulario del corpus.
`frontend/`	Interfaz web (React + TypeScript) y pruebas de extremo a extremo (`e2e/`).
`tests/`	Pruebas automatizadas (`pytest`) y la referencia manual de metadatos.
`scripts/`	Generación de los embeddings de los modelos de IA.
`main.py`	Menú por consola.
`Informe_Final_Analisis_Algoritmos.pdf`	Este documento; la aplicación lo ofrece para descarga en `/informe.pdf`.
`Dockerfile, docker-compose.yml, deploy/`	Despliegue: imagen, servicios y proxy inverso con HTTPS.

2.4. Flujo de una solicitud

Cuando el usuario compara artículos en la interfaz ocurre lo siguiente: (1) el navegador envía `POST /api/similarity/compare` con los identificadores; (2) la API valida la solicitud con Pydantic y delega en `SimilarityEngine`; (3) el motor, que al iniciar preprocesó los 20 abstracts y construyó el modelo TF-IDF, ejecuta los seis algoritmos sobre cada par; (4) la respuesta JSON incluye los valores, las matrices y los tiempos, y (5) la interfaz los dibuja (gráfica de barras, mapa de calor y tabla). El agrupamiento y la búsqueda de patrones siguen el mismo esquema (tabla 33, sección 8).

3. Corpus y extracción de metadatos

3.1. Fuente de información

El corpus está formado por los 20 artículos en PDF entregados por el docente, publicados entre 2019 y 2024 en 7 revistas (Elsevier, Springer, Emerald, Wiley y MDPI). Todos son revisiones de literatura sobre IA en educación; esta homogeneidad temática es relevante para interpretar los resultados de similitud y de agrupamiento. Los abstracts tienen en promedio 200 palabras (entre 132 y 257); en conjunto suman 68 autores distintos. La tabla 35 del anexo A lista el corpus completo.

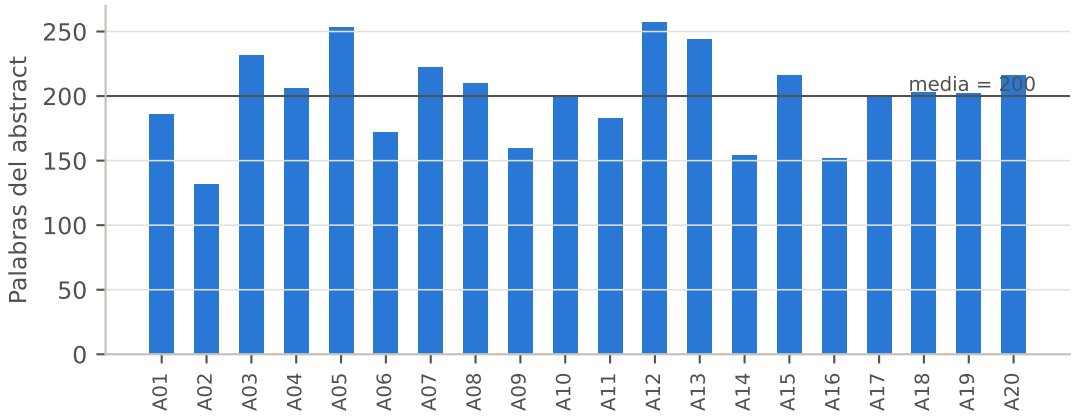


Figura 2: Longitud (en palabras) del abstract de cada artículo.

3.2. Algoritmo de extracción

Los PDF provienen de cinco editoriales con diagramaciones muy distintas, por lo que una regla única como «el texto con la fuente más grande es el título» falla: en Elsevier y MDPI el nombre de la revista usa una fuente *mayor* que la del título; en Emerald el título se intercala con un encabezado corrido; en Wiley y MDPI una barra lateral ocupa el mismo rango vertical que el abstract. Por ello el extractor (`bibliometria/extraction/pdf_extractor.py`) trabaja sobre la *geometría* de la primera página: PyMuPDF entrega cada línea con su caja delimitadora, tamaño de fuente y banderas (negrita, superíndice), y sobre ese insumo se aplican las siguientes reglas propias:

1. **Título.** Las líneas se agrupan por tamaño de fuente en bloques verticalmente contiguos y se elige el bloque más parecido al nombre del archivo (`"N. Título.pdf"`). La semejanza es $\mathrm{Dice}_{LCS}(a,b) = \frac{2\,|LCS(a,b)|}{|a|+|b|}$ sobre las secuencias de tokens, calculada con programación dinámica.
2. **Autores.** Primer bloque bajo el título con aspecto de lista de nombres. Los superíndices (marcas de afiliación) se descartan por su tamaño de fuente y su bandera, y los nombres se separan por «,», «and», «&», «|» y «·». Cuando dos fragmentos están separados por un hueco horizontal (iconos ORCID) se inserta un espacio.
3. **Abstract.** Desde el encabezado «Abstract» (o «A B S T R A C T») se recorren las líneas de la *misma columna* hasta un marcador de fin (*Keywords*, *Introduction*, ©, *Received*…). Las barras laterales y la columna de *Article info* quedan fuera por su posición horizontal; los resúmenes estructurados (Emerald) se normalizan a la forma «Purpose: …».
4. **Guiones de fin de línea.** Para decidir si `edu-|cation` se une (`education`) o se conserva (`AI-|driven`), se comparan las frecuencias de ambas formas en el texto completo del propio documento.
5. **Palabras clave, año, revista y DOI** se obtienen con patrones sobre los encabezados y pies de página de cada editorial.

extraction/pdf_extractor.py — decisión sobre los guiones de fin de línea

```python
def _join_hyphen(self, left: str, right: str) -> str:
    prefix = left[:-1].split(" ")[-1]
    suffix = right.split(" ")[0]
    p = prefix.strip(".,;:()[]\"'").lower()
    s = suffix.strip(".,;:()[]\"'").lower()
    joined_count = self.words.get(p + s, 0)
    hyph_count = self.hyphenated.get(p + "-" + s, 0)
    keep = False
    if hyph_count > joined_count:
        keep = True
    elif hyph_count == joined_count == 0 and (suffix[:1].isupper() or prefix.isupper()):
        keep = True  # p. ej. "AI-" + "Driven", "COVID-" + "19"
    if keep:
        return left[:-1] + "-" + right
    return left[:-1] + right
```

3.3. Validación

Para medir la calidad de la extracción se construyó una *referencia* verificada manualmente (`tests/fixtures/ground_truth_metadata.json`), comparando cada campo con la imagen de la página. El extractor reproduce **los 140 campos** (7 campos × 20 artículos) de forma exacta, tras normalizar comillas y guiones tipográficos; la prueba `tests/test_extraction.py` lo verifica en cada ejecución. El tiempo medio de extracción es de 72 ms por PDF.

Resultado de la extracción

Título, autores y abstract correctos en 20/20 artículos; año, revista, DOI y palabras clave correctos en 20/20. El corpus resultante se guarda en `data/processed/corpus.json` y se puede regenerar con `python -m bibliometria.extraction.build_corpus` (opción 1 del menú).

4. Preprocesamiento del texto

Los requerimientos 1 y 2 parten de la misma transformación del abstract, implementada en `bibliometria/preprocessing/`:

1. **Normalización**: reemplazo de ligaduras tipográficas (el par *fi* codificado como un solo carácter Unicode se convierte en dos letras), guiones suaves y espacios especiales; colapso de espacios. Costo $O(n)$.
2. **Tokenización**: autómata que recorre el texto carácter por carácter y emite secuencias maximales de caracteres alfanuméricos, en minúsculas. Costo $O(n)$.
3. **Filtrado**: eliminación de las 179 *stopwords* estándar del inglés, de conectores académicos genéricos, de números y de tokens de un carácter.
4. **Stemming de Porter** (1980) [7]: reducción de cada palabra a su raíz en cinco pasos de reglas de sufijos condicionadas por la *medida* m de la palabra, definida por su forma $[C](VC)^m[V]$ (C = consonantes, V = vocales). Por ejemplo, `education`→`educ` y `learning`→`learn`. Costo $O(L)$ por palabra.

El stemmer es una implementación propia. Como verificación, una prueba de `tests/test_preprocessing.py` lo compara con el modo `ORIGINAL_ALGORITHM` de NLTK sobre todo el vocabulario del corpus, y además se contrastó con las 14 768 palabras distintas de los 20 PDF completos: no hubo ninguna diferencia.

preprocessing/porter.py — cálculo de la medida m

```python
def measure(self, stem: str) -> int:
    m = 0
    prev_vowel = False
    for i in range(len(stem)):
        consonant = self._is_consonant(stem, i)
        if consonant and prev_vowel:
            m += 1
        prev_vowel = not consonant
    return m
```

Ejemplo. Primera oración del abstract de A11:

Original *Generative artificial intelligence provides both opportunities and challenges for higher education.*

Tokens (11) `generative | artificial | intelligence | provides | both | opportunities | and | challenges | for | higher | education`

Sin stopwords (8) `generative | artificial | intelligence | provides | opportunities | challenges | higher | education`

Raíces (8) `gener | artifici | intellig | provid | opportun | challeng | higher | educ`

Sobre el abstract completo, A11 pasa de 188 tokens a 115 palabras de contenido (72 raíces distintas) y A16 de 153 a 93 (65 raíces distintas). Los modelos de IA no usan el stemming: GloVe necesita palabras completas para consultar sus vectores y Sentence-BERT recibe el abstract original.

5. Requerimiento 1: Análisis de similitud textual

5.1. Flujo general del comparador

La aplicación permite seleccionar dinámicamente dos o más artículos (en la interfaz, con un selector con búsqueda; en la consola, escribiendo sus identificadores). Para cada par (A, B) de la selección, `SimilarityEngine.compare` calcula seis similitudes normalizadas en $[0, 1]$ y devuelve también la matriz de cada algoritmo y su tiempo de cálculo. Los cuatro algoritmos clásicos operan sobre la secuencia de raíces de la sección 4; Levenshtein admite además la comparación por caracteres. `SimilarityEngine.explain` produce el desglose matemático de un par: matrices, alineamientos, conjuntos, pesos y vectores. Las tablas de esta sección se generaron con esa misma función.

Tabla 3: Algoritmos del Requerimiento 1.

Algoritmo	Enfoque	Similitud	Complejidad
Levenshtein	Programación dinámica	$1 - \frac{d(A,B)}{\max(\lvert A\rvert,\lvert B\rvert)}$	$O(nm)$
Needleman-Wunsch	Programación dinámica	$\frac{\#\text{coincidencias}}{L_{\text{alin}}}$	$O(nm)$
Jaccard	Conjuntos	$\frac{\lvert A\cap B\rvert}{\lvert A\cup B\rvert}$	$O(n+m)$
Coseno TF-IDF	Espacio vectorial	$\frac{\vec{w}_A\cdot\vec{w}_B}{\lVert\vec{w}_A\rVert\lVert\vec{w}_B\rVert}$	$O(n+m)$
GloVe + coseno	IA: embeddings estáticos	$\cos(\vec{v}_A,\vec{v}_B)$	$O((n+m)d)$
Sentence-BERT + coseno	IA: embeddings contextuales	$\cos(\vec{s}_A,\vec{s}_B)$	$O(L^2 d)$ por texto

5.2. Caso de estudio

Se eligieron dos abstracts del mismo subtema, IA generativa y evaluación, escritos con vocabulario y estructura diferentes. Es el escenario que mejor contrasta los enfoques léxicos con los semánticos.

A11 — A scoping review on how generative artificial intelligence transforms assessment in higher education

Generative artificial intelligence provides both opportunities and challenges for higher education. Existing literature has not properly investigated how this technology would impact assessment in higher education. This scoping review took a forward-thinking approach to investigate how generative artificial intelligence transforms assessment in higher education. We used the PRISMA extension for scoping reviews to select articles for review and report the results. In the screening, we retrieved 969 articles and selected 32 empirical studies for analysis. Most of the articles were published in 2023. We used three levels—students, teachers, and institutions—to analyses the articles. Our results suggested that assessment should be transformed to cultivate students' self-regulated learning skills, responsible learning, and integrity. To successfully transform assessment in higher education, the review suggested that (i) teacher professional development activities for assessment, AI, and digital literacy should be provided, (ii) teachers' beliefs about human and AI assessment should be strengthened, and (iii) teachers should be innovative and holistic in their teaching to reflect the assessment transformation. Educational institutions are recommended to review and rethink their assessment policies, as well as provide more interdisciplinary programs and teaching.

A16 — Generative Artificial Intelligence in Tertiary Education: Assessment Redesign Principles and Considerations

The emergence of generative artificial intelligence (AI) such as ChatGPT has sparked significant assessment concerns within tertiary education. Assessment concerns have largely revolved around academic integrity issues among students, such as plagiarism and cheating. Nonetheless, it is also critical to consider that generative AI models trained on information retrieved from the Internet could produce biased and discriminatory outputs, and hallucination issues in large language models upon which generative AI acts provide made-up and untruthful outputs. This article considers the affordances and challenges of generative AI specific to assessments within tertiary education. It illustrates considerations for assessment redesign with the existence of generative AI and proposes the Against, Avoid and Adopt (AAA) principle to rethink and redesign assessments. It argues that more generative AI tools will emerge exponentially, and hence, engaging in an arms race against generative AI and policing the use of these technologies may not address the fundamental issues in assessments.

Tras el preprocesamiento, A tiene $|A| = 115$ raíces y B tiene $|B| = 93$. Las primeras raíces de cada uno son:

- A: `gener artifici intellig provid opportun challeng higher educ …`
- B: `emerg gener artifici intellig ai chatgpt spark signific …`

5.3. Algoritmo clásico 1: distancia de Levenshtein

Definición. La distancia de edición [1] $d(A, B)$ es el número mínimo de inserciones, eliminaciones y sustituciones que transforman A en B. Se calcula con programación dinámica (Wagner y Fischer, 1974) [2] llenando una matriz D de $(n + 1) \times (m + 1)$, donde $D[i][j]$ es la distancia entre los prefijos $A[1..i]$ y $B[1..j]$:

$$D[i][0] = i, \qquad D[0][j] = j, \qquad D[i][j] = \text{mín} \begin{cases} D[i-1][j] + 1 & \text{(eliminación)} \\ D[i][j-1] + 1 & \text{(inserción)} \\ D[i-1][j-1] + [a_i \neq b_j] & \text{(sustitución)} \end{cases} \tag{1}$$

La subestructura óptima se justifica así: en una edición óptima la última operación sólo puede ser una de las tres, y el resto debe ser a su vez óptimo para los prefijos. La similitud normalizada es

$$\text{sim}_{\text{Lev}}(A, B) = 1 - \frac{d(A, B)}{\text{máx}(|A|, |B|)} \in [0, 1]. \tag{2}$$

Algoritmo 1 Distancia de Levenshtein con dos filas

Entrada: secuencias A (longitud n) y B (longitud m)

```
prev ← [0, 1, ..., m]
para i ← 1 hasta n hacer
    cur[0] ← i
    para j ← 1 hasta m hacer
        cur[j] ← mín(prev[j] + 1, cur[j − 1] + 1, prev[j − 1] + [a_i ≠ b_j])
    fin para
    prev ← cur
fin para
retornar prev[m]
```

Complejidad. Se llenan $n \cdot m$ celdas con trabajo $O(1)$: tiempo $\Theta(nm)$. Como cada fila sólo depende de la anterior, la distancia requiere memoria $O(\text{mín}(n, m))$; la matriz completa, necesaria para el *traceback* y la demostración, ocupa $O(nm)$.

Demostración paso a paso. La tabla 4 muestra el llenado de la matriz para las primeras **8 raíces** de cada abstract. Las celdas sombreadas en azul forman una ruta óptima obtenida con *traceback* desde $D[8][8]$, y las verdes marcan raíces iguales.

Tabla 4: Matriz D de Levenshtein sobre las primeras 8 raíces de A11 (filas) y A16 (columnas).

	ε	emerg	gener	artifici	intellig	ai	chatgpt	spark	signific
ε	**0**	**1**	2	3	4	5	6	7	8
gener	1	1	**1**	2	3	4	5	6	7
artifici	2	2	2	**1**	2	3	4	5	6
intellig	3	3	3	2	**1**	2	3	4	5
provid	4	4	4	3	**2**	2	3	4	5
opportun	5	5	5	4	3	**3**	3	4	5
challeng	6	6	6	5	4	4	**4**	4	5
higher	7	7	7	6	5	5	5	**5**	5
educ	8	8	8	7	6	6	6	6	**6**

Algunas celdas calculadas con la recurrencia:

$$D[1][1] = \text{mín}\{D[0][1] + 1,\ D[1][0] + 1,\ D[0][0] + 1\} = \text{mín}\{2,\ 2,\ 1\} = 1 \quad (\texttt{gener} \text{ vs } \texttt{emerg})$$

$$D[2][2] = \text{mín}\{D[1][2] + 1,\ D[2][1] + 1,\ D[1][1] + 1\} = \text{mín}\{2,\ 3,\ 2\} = 2 \quad (\texttt{artifici} \text{ vs } \texttt{gener})$$

$$D[3][3] = \text{mín}\{D[2][3] + 1,\ D[3][2] + 1,\ D[2][2] + 1\} = \text{mín}\{2,\ 4,\ 3\} = 2 \quad (\texttt{intellig} \text{ vs } \texttt{artifici})$$

La ruta óptima corresponde a la secuencia de operaciones: +emerg, gener (=), artifici (=), intellig (=), −provid, opportun→ai, challeng→chatgpt, higher→spark, educ→signific. La distancia de la ventana es $D[8][8] = 6$.

Resultado sobre los abstracts completos

Por raíces: $d(A, B) = 107$ con $|A| = 115$ y $|B| = 93$, luego $\text{sim}_{\text{Lev}} = 1 - \frac{107}{115} = \mathbf{0.0696}$. Por caracteres: $d = 972$ con longitudes 1379 y 1088, luego sim $= 1 - \frac{972}{1379} = 0.2951$ (matriz de más de 1.5 millones de celdas).

similarity/levenshtein.py

```python
def levenshtein_distance(a: Sequence, b: Sequence) -> int:
    n, m = len(a), len(b)
    if n < m:  # la fila se construye sobre la secuencia más corta
        a, b, n, m = b, a, m, n
    if m == 0:
        return n
    prev = list(range(m + 1))
    for i in range(1, n + 1):
        cur = [i] + [0] * m
        ai = a[i - 1]
        for j in range(1, m + 1):
            best = prev[j - 1] if ai == b[j - 1] else prev[j - 1] + 1  # sustitución/coincidencia
            deletion = prev[j] + 1
            if deletion < best:
                best = deletion
            insertion = cur[j - 1] + 1
            if insertion < best:
                best = insertion
            cur[j] = best
        prev = cur
    return prev[m]
```

5.4. Algoritmo clásico 2: alineamiento global de Needleman-Wunsch

Definición. Needleman y Wunsch (1970) [3] propusieron alinear globalmente dos secuencias maximizando una puntuación: $s(x, y) = +1$ si $x = y$ (coincidencia), -1 si $x \neq y$ (discrepancia) y $g = -1$ por cada hueco. La matriz F cumple

$$F[i][0] = i\,g, \quad F[0][j] = j\,g, \quad F[i][j] = \max\{F[i-1][j-1]+s(a_i, b_j),\ F[i-1][j]+g,\ F[i][j-1]+g\}. \tag{3}$$

$F[n][m]$ es la puntuación del mejor alineamiento, que se reconstruye desde (n, m) siguiendo en cada celda la opción que produjo el máximo. Como similitud se usa la *identidad* del alineamiento, una medida estándar en bioinformática:

$$\mathrm{sim}_{NW}(A, B) = \frac{\#\text{coincidencias}}{\text{longitud del alineamiento}}. \tag{4}$$

A diferencia de Levenshtein, que *minimiza un costo*, Needleman-Wunsch *maximiza una puntuación* configurable. Esto permite, por ejemplo, premiar más las coincidencias o castigar más los huecos.

Demostración paso a paso. Para las mismas ventanas (tabla 5), la primera celda se calcula como $F[1][1] = \max\{F[0][0] + s(\texttt{gener}, \texttt{emerg}),\ F[0][1] + g,\ F[1][0] + g\} = \max\{0 - 1,\ -1 - 1,\ -1 - 1\} = -1$ y la puntuación de la ventana es $F[8][8] = -3$.

Tabla 5: Matriz F de Needleman-Wunsch (coincidencia $+1$, discrepancia -1, hueco -1).

	ε	emerg	gener	artifici	intellig	ai	chatgpt	spark	signific
ε	0	-1	-2	-3	-4	-5	-6	-7	-8
gener	-1	-1	0	-1	-2	-3	-4	-5	-6
artifici	-2	-2	-1	1	0	-1	-2	-3	-4
intellig	-3	-3	-2	0	2	1	0	-1	-2
provid	-4	-4	-3	-1	1	1	0	-1	-2
opportun	-5	-5	-4	-2	0	0	0	-1	-2
challeng	-6	-6	-5	-3	-1	-1	-1	-1	-2
higher	-7	-7	-6	-4	-2	-2	-2	-2	-2
educ	-8	-8	-7	-5	-3	-3	-3	-3	-3

```
--     gener  artifici  intellig  provid  opportun  challeng  higher  educ  exist  literatur  properli  investig  technologi  impact  assess  --       higher  educ  scope  review  took
         |       |         |                                                          ·         ·          ·        ·       |              ·       |
emerg  gener  artifici  intellig  --      --        --        --      --    --     --         ai        chatgpt   spark       signific  assess  concern  tertiari  educ  --     --       --
```

Figura 3: Primeras 22 columnas del alineamiento global óptimo de los abstracts completos («|» coincidencia, «·» discrepancia, «–» hueco).

Resultado sobre los abstracts completos

$F[n][m] = -97$; el alineamiento tiene longitud 117 con 10 coincidencias, 81 discrepancias y 26 huecos, luego $\text{sim}_{NW} = \frac{10}{117} = \mathbf{0.0855}$.

Complejidad. Tiempo $\Theta(nm)$ y memoria $\Theta(nm)$, porque el *traceback* necesita la matriz completa; la reconstrucción del alineamiento cuesta $O(n+m)$.

similarity/needleman_wunsch.py

```python
def needleman_wunsch(
    a: Sequence,
    b: Sequence,
    match: int = 1,
    mismatch: int = -1,
    gap: int = -1,
    keep_matrix: bool = False,
) -> NWResult:
    n, m = len(a), len(b)
    F = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        F[i][0] = i * gap
    for j in range(1, m + 1):
        F[0][j] = j * gap

    for i in range(1, n + 1):
        ai = a[i - 1]
        row, up = F[i], F[i - 1]
        for j in range(1, m + 1):
            diag = up[j - 1] + (match if ai == b[j - 1] else mismatch)
            best = diag
            delete = up[j] + gap
            if delete > best:
                best = delete
            insert = row[j - 1] + gap
            if insert > best:
```

```
                best = insert
            row[j] = best

    # Traceback: se prefiere la diagonal, luego el hueco en B y luego en A.
    aligned_a: list = []
    aligned_b: list = []
    matches = mismatches = gaps = 0
    i, j = n, m
    while i > 0 or j > 0:
        if i > 0 and j > 0:
            s = match if a[i - 1] == b[j - 1] else mismatch
            if F[i][j] == F[i - 1][j - 1] + s:
                aligned_a.append(a[i - 1])
                aligned_b.append(b[j - 1])
                if s == match and a[i - 1] == b[j - 1]:
                    matches += 1
                else:
                    mismatches += 1
                i, j = i - 1, j - 1
                continue
        if i > 0 and F[i][j] == F[i - 1][j] + gap:
            aligned_a.append(a[i - 1])
            aligned_b.append(GAP)
            gaps += 1
            i -= 1
        else:
            aligned_a.append(GAP)
            aligned_b.append(b[j - 1])
            gaps += 1
            j -= 1
    aligned_a.reverse()
    aligned_b.reverse()
    return NWResult(
        score=F[n][m],
        aligned_a=aligned_a,
        aligned_b=aligned_b,
        matches=matches,
        mismatches=mismatches,
        gaps=gaps,
        matrix=F if keep_matrix else None,
    )
```

5.5. Algoritmo clásico 3: coeficiente de Jaccard

Definición. Siguiendo a Jaccard [4], sean $T(A)$ y $T(B)$ los *conjuntos* de raíces (sin repetición) de cada abstract:

$$J(A,B) = \frac{|T(A) \cap T(B)|}{|T(A) \cup T(B)|} = \frac{|T(A) \cap T(B)|}{|T(A)| + |T(B)| - |T(A) \cap T(B)|}. \tag{5}$$

Ignora el orden y la frecuencia de los términos; sólo mide el vocabulario compartido.

Demostración. $|T(A)| = 72$, $|T(B)| = 65$ y la intersección tiene 16 raíces: `ai`, `articl`, `artifici`, `assess`, `challeng`, `educ`, `exist`, `gener`, `integr`, `intellig`, `provid`, `rethink`, `retriev`, `student`, `technologi`, `us`. Por tanto

$$J(A,B) = \frac{16}{72 + 65 - 16} = \frac{16}{121} = \mathbf{0.1322}. \tag{6}$$

Complejidad. Los conjuntos se construyen como tablas hash (diccionarios) en $O(n+m)$; la intersección recorre el conjunto menor con búsquedas $O(1)$ en promedio. Total: $O(n+m)$ esperado.

similarity/jaccard.py

```python
def jaccard(a: Iterable[str], b: Iterable[str]) -> JaccardResult:
    set_a = _hash_set(a)
    set_b = _hash_set(b)
    small, large = (set_a, set_b) if len(set_a) <= len(set_b) else (set_b, set_a)
    intersection: list[str] = []
    for term in small:
        if term in large:
            intersection.append(term)
    only_a = [t for t in set_a if t not in set_b]
    only_b = [t for t in set_b if t not in set_a]
    union_size = len(set_a) + len(set_b) - len(intersection)
    intersection.sort()
    only_a.sort()
    only_b.sort()
    return JaccardResult(intersection=intersection, only_a=only_a, only_b=only_b,
     union_size=union_size)
```

5.6. Algoritmo clásico 4: similitud del coseno sobre TF-IDF

Definición. En el modelo de espacio vectorial [5, 6], cada abstract se representa como un vector en el espacio de los $|V| = 746$ términos del corpus. Para un término t y un documento d, con $N = 20$ documentos:

$$tf(t,d) = \text{número de apariciones de } t \text{ en } d, \qquad df(t) = \#\{d : t \in d\}, \tag{7}$$

$$idf(t) = \ln \frac{1+N}{1+df(t)} + 1, \qquad w(t,d) = tf(t,d) \cdot idf(t), \qquad \hat{w}(t,d) = \frac{w(t,d)}{\|\vec{w}_d\|}. \tag{8}$$

El *idf* suavizado evita divisiones por cero y no anula del todo los términos presentes en todos los documentos; es la misma convención de scikit-learn, lo que permitió validar la implementación contra esa librería (tabla 34). La similitud es el coseno del ángulo θ entre los vectores:

$$\cos\theta = \frac{\vec{w}_A \cdot \vec{w}_B}{\|\vec{w}_A\|\,\|\vec{w}_B\|} = \sum_t \hat{w}(t,A)\,\hat{w}(t,B). \tag{9}$$

Sólo los términos compartidos aportan al producto punto.

Demostración paso a paso. El término de mayor aporte es `assess` (*assessment*), que aparece en $df = 11$ de los 20 abstracts: $idf = \ln \frac{21}{1+11} + 1 = 1.5596$. En A11 aparece $tf = 8$ veces, luego $w = 8 \times 1.5596 = 12.4769$. La tabla 6 lista los 16 términos compartidos.

Tabla 6: Pesos TF-IDF de los términos compartidos por A11 y A16 ($\hat{w}$: pesos normalizados con la norma L2).

Raíz	Palabra	tf_A	tf_B	df	idf	$\hat{w}_A$	$\hat{w}_B$	$\hat{w}_A\hat{w}_B$
assess	assessment	8	6	11	1.5596	0.3773	0.2925	0.1103
gener	generative	2	7	9	1.7419	0.1053	0.3811	0.0401
ai	ai	2	7	19	1.0488	0.0634	0.2295	0.0146
educ	education	5	2	18	1.1001	0.1663	0.0688	0.0114
articl	articles	4	1	10	1.6466	0.1991	0.0515	0.0103
rethink	rethink	1	1	2	2.9459	0.0891	0.0921	0.0082
retriev	retrieved	1	1	2	2.9459	0.0891	0.0921	0.0082
exist	existing	1	1	3	2.6582	0.0804	0.0831	0.0067
student	students	2	1	8	1.8473	0.1117	0.0577	0.0065
provid	provide	3	1	14	1.3365	0.1212	0.0418	0.0051
integr	integrity	1	1	5	2.2528	0.0681	0.0704	0.0048
challeng	challenges	1	1	6	2.0986	0.0635	0.0656	0.0042
us	used	2	1	15	1.2719	0.0769	0.0398	0.0031
technologi	technology	1	1	12	1.4796	0.0447	0.0462	0.0021
artifici	artificial	2	1	20	1.0000	0.0605	0.0313	0.0019
intellig	intelligence	2	1	20	1.0000	0.0605	0.0313	0.0019

Con los pesos sin normalizar, $\vec{w}_A \cdot \vec{w}_B = 253.093$, $\|\vec{w}_A\| = 33.073$ y $\|\vec{w}_B\| = 31.993$:

$$\cos\theta = \frac{253.093}{33.073 \times 31.993} = \mathbf{0.2392} \qquad \Longrightarrow \qquad \theta = \arccos(0.2392) = 76.16^\circ. \tag{10}$$

Comprobación: la suma de la última columna de la tabla 6 es 0.2392.

Complejidad. Construir el modelo cuesta $O(\sum_d |d|)$ (una pasada por los tokens del corpus); con vectores dispersos (diccionarios), el coseno de un par cuesta $O(n+m)$.

similarity/tfidf.py — ajuste de *df* e *idf*

```python
def __init__(self, documents: list[list[str]], smooth_idf: bool = True) -> None:
    self.n_docs = len(documents)
    self.smooth_idf = smooth_idf
    self.df: dict[str, int] = {}
    for doc in documents:
        seen: dict[str, bool] = {}
        for term in doc:
            if term not in seen:
                seen[term] = True
                self.df[term] = self.df.get(term, 0) + 1
    self.idf: dict[str, float] = {term: self._idf_value(df) for term, df in self.df.items()}
```

similarity/vector_math.py — coseno disperso

```python
def sparse_cosine(u: dict[str, float], v: dict[str, float]) -> float:
    nu, nv = sparse_norm(u), sparse_norm(v)
    if nu == 0.0 or nv == 0.0:
        return 0.0
    value = sparse_dot(u, v) / (nu * nv)
    return 1.0 if value > 1.0 else value
```

5.7. Modelo de IA 1: GloVe con promedio de embeddings

Modelo. GloVe (Pennington, Socher y Manning, 2014) [9] aprende un vector $\vec{v}_w \in \mathbb{R}^d$ por palabra factorizando la matriz de coocurrencias X de un corpus masivo, minimizando

$$J = \sum_{i,j} f(X_{ij}) \left(\vec{v}_i^{\top} \tilde{\vec{v}}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2, \tag{11}$$

de modo que el producto punto de dos vectores aproxima el logaritmo de su coocurrencia. Se usó el modelo preentrenado `glove-wiki-gigaword-100`: 400 000 palabras y $d = 100$, entrenado sobre Wikipedia 2014 y Gigaword 5. De sus 400 000 vectores se conservan los de las 980 palabras que aparecen en el corpus (`data/models/`). Es un modelo de la misma familia que Word2Vec [8] y FastText, sugeridos en el enunciado.

Vector del documento. Como GloVe asigna vectores a palabras y no a textos, el abstract se representa con el promedio (*mean pooling*) de los vectores de sus palabras de contenido W_d (sin stopwords ni stemming):

$$\vec{e}(d) = \frac{1}{|W_d|} \sum_{w \in W_d} \vec{v}_w, \qquad \mathrm{sim}_{\mathrm{GloVe}}(A, B) = \cos(\vec{e}(A), \vec{e}(B)). \tag{12}$$

Demostración. A11 aporta 115 palabras con vector y A16 92. Hubo palabras fuera de vocabulario: `chatgpt`. Es un término posterior al entrenamiento de GloVe (2014), una limitación importante en un corpus sobre IA generativa. La tabla 7 muestra las primeras 6 de las 100 dimensiones.

Tabla 7: Vectores GloVe de algunas palabras de A11 y vectores promedio de ambos documentos (primeras 6 de 100 dimensiones).

	x_1	x_2	x_3	x_4	x_5	x_6	
`generative`	0.3524	0.7799	0.1549	0.1126	-0.1590	0.2725	⋯
`artificial`	0.3455	0.3144	-0.0313	0.6368	0.2727	-0.6197	⋯
`intelligence`	-0.3110	-0.4329	0.7773	-0.3111	0.0529	-0.8502	⋯
`provides`	-0.3990	0.7222	0.3576	0.4420	0.9607	-0.1499	⋯
$\vec{e}(A)$	-0.0703	0.2667	-0.0590	0.2133	0.0179	-0.0965	⋯
$\vec{e}(B)$	-0.0835	0.2641	0.1889	0.1685	-0.0400	-0.0912	⋯

$$\cos\theta = \frac{\vec{e}_A \cdot \vec{e}_B}{\|\vec{e}_A\|\|\vec{e}_B\|} = \frac{7.5934}{3.2076 \times 2.5323} = \mathbf{0.9349} \quad (\theta = 20.79^\circ), \qquad d_E(\vec{e}_A, \vec{e}_B) = 1.2305. \tag{13}$$

Complejidad. Consultar los vectores y promediarlos cuesta $O(|W| \cdot d)$; el coseno, $O(d)$.

similarity/embeddings.py — vector promedio

```python
def embed(self, text: str) -> EmbeddingResult:
    used, oov, vecs = [], [], []
    for w in content_words(text):
        v = self.vectors.get(w)
        if v is None:
            oov.append(w)
        else:
            used.append(w)
            vecs.append(v)
    vector = mean_vector(vecs) if vecs else [0.0] * self.dim
    return EmbeddingResult(vector=vector, used_tokens=used, oov_tokens=oov)
```

5.8. Modelo de IA 2: Sentence-BERT

Modelo. Sentence-BERT (Reimers y Gurevych, 2019) [11] adapta un transformer preentrenado para producir un embedding por oración o párrafo, entrenándolo con pares de textos semánticamente relacionados (aprendizaje contrastivo). Se usó `all-MiniLM-L6-v2` [12, 30]: 6 capas transformer, dimensión 384, entrenado sobre más de mil millones de pares. El cálculo tiene tres etapas:

1. **Tokenización WordPiece**: el texto se divide en subpalabras (`chatgpt`→`chat`, `##gp`, `##t`), así que no existen palabras fuera de vocabulario.
2. **Autoatención** [10]: en cada capa, cada token combina la información de todos los demás:

$$\text{Atención}(Q,K,V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \tag{14}$$

   de modo que el vector de una palabra depende de su contexto (embedding *contextual*).
3. **Mean pooling y normalización**: $\vec{s}(d) = \frac{\bar{\vec{h}}}{\|\bar{\vec{h}}\|}$, donde $\bar{\vec{h}}$ es el promedio de los vectores de salida de los tokens.

Configuración. Por defecto el modelo trunca la entrada a 256 tokens, lo que habría recortado 13 de los 20 abstracts (el más largo tiene 364 tokens). Por eso se fijó `max_seq_length = 512`, el límite de posiciones de la arquitectura, y ningún abstract se trunca. A11 tiene 235 tokens y A16, 196.

Tabla 8: Embeddings Sentence-BERT de A11 y A16 (primeras 8 de 384 dimensiones; vectores con norma 1).

	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	
$\vec{s}(A)$	-0.0728	0.0169	0.0063	0.0105	-0.0377	-0.0087	-0.0471	-0.0015	⋯
$\vec{s}(B)$	-0.0985	-0.0041	0.0579	-0.0224	0.0035	-0.0115	-0.0235	-0.0367	⋯

Como los vectores están normalizados, el coseno coincide con el producto punto:

$$\cos\theta = \vec{s}_A \cdot \vec{s}_B = \mathbf{0.7356} \quad (\theta = 42.65°), \qquad d_E = \sqrt{2 - 2\cos\theta} = 0.7272. \tag{15}$$

Complejidad. La autoatención cuesta $O(L^2 d)$ por capa, con L tokens. La codificación se hace una vez por abstract (0.2 s para los 20 en CPU) y se guarda; en la aplicación sólo se calcula el coseno, en $O(d)$.

similarity/vector_math.py — coseno denso (usado por ambos modelos de IA)

```python
def cosine(u: Sequence[float], v: Sequence[float]) -> float:
    nu, nv = norm(u), norm(v)
    if nu == 0.0 or nv == 0.0:
        return 0.0
    value = dot(u, v) / (nu * nv)
    # Corrige errores de redondeo fuera de [-1, 1].
    if value > 1.0:
        return 1.0
    if value < -1.0:
        return -1.0
    return value
```

5.9. Análisis crítico: enfoques clásicos frente a modelos de IA

Resultados del caso de estudio

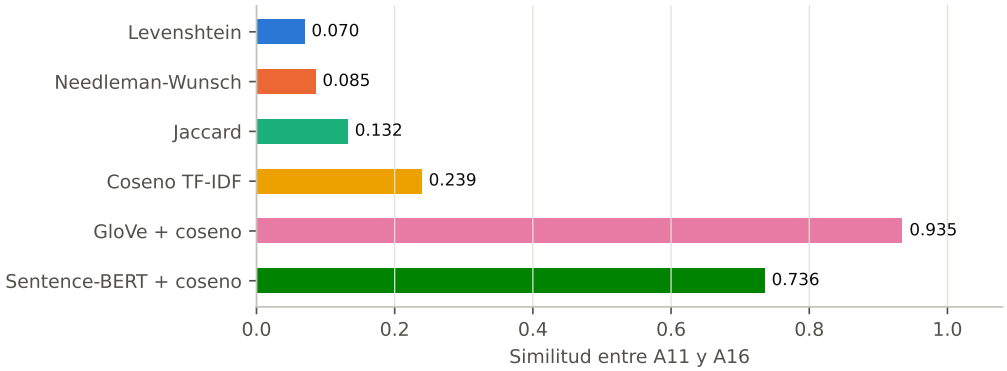


Figura 4: Similitud entre A11 y A16 según los seis algoritmos.

Los resultados se ordenan en tres niveles:

- **Programación dinámica (0.0696 y 0.0855)**: valores casi nulos. Ambos algoritmos exigen que los términos coincidan *en el mismo orden*; dos autores que escriben sobre el mismo tema con frases distintas producen secuencias que requieren editar casi todo. Por caracteres, Levenshtein sube a 0.2951, pero ese aumento no indica afinidad temática: lo produce el alfabeto común del inglés, y los 190 pares del corpus quedan todos entre 0.22 y 0.32 (tabla 9).
- **Conjuntos y espacio vectorial (0.1322 y 0.2392)**: detectan el vocabulario compartido (*generative*, *assessment*, *education*, *integrity*…) sin importar el orden. TF-IDF da más peso a los términos poco frecuentes en el corpus y por eso supera a Jaccard. Aun así, $\theta = 76.16°$: los vectores son casi ortogonales porque cada abstract usa muchos términos que el otro no usa.
- **Modelos de IA (0.9349 y 0.7356)**: reconocen que ambos textos tratan de lo mismo, la transformación de la evaluación por la IA generativa, aunque usen palabras distintas (*higher education* frente a *tertiary education*, *transforms* frente a *redesign*). Sentence-BERT ubica este par como el **tercero más similar de los 190** del corpus. GloVe le asigna 0.9349, un valor que parece alto pero que en su escala es bajo: es el par número 179 de 190. Los dos modelos de IA discrepan, y el análisis siguiente muestra por qué el de GloVe es poco informativo.

Comportamiento sobre los 190 pares del corpus

Un solo par no basta para juzgar un algoritmo. La tabla 9 y las figuras 5 y 6 resumen las matrices completas de 20 × 20.

Tabla 9: Estadísticas de cada algoritmo sobre los 190 pares (media, desviación estándar, rango, par más similar, valor y posición del caso de estudio, y tiempo total de cálculo).

Algoritmo	Media	D.E.	Mín.	Máx.	Par más similar	Caso	Posición	ms
Levenshtein	0.056	0.023	0.013	0.118	A01–A02	0.070	51/190	183
Needleman-Wunsch	0.071	0.024	0.026	0.150	A02–A09	0.085	46/190	212
Jaccard	0.122	0.032	0.039	0.214	A02–A11	0.132	66/190	3
Coseno TF-IDF	0.149	0.062	0.032	0.318	A08–A10	0.239	21/190	1
GloVe + coseno	0.958	0.014	0.923	0.984	A17–A20	0.935	179/190	1
Sentence-BERT + coseno	0.554	0.089	0.349	0.810	A06–A09	0.736	3/190	3
Levenshtein (caracteres)	0.272	0.023	0.221	0.317	—	0.295	—	33556

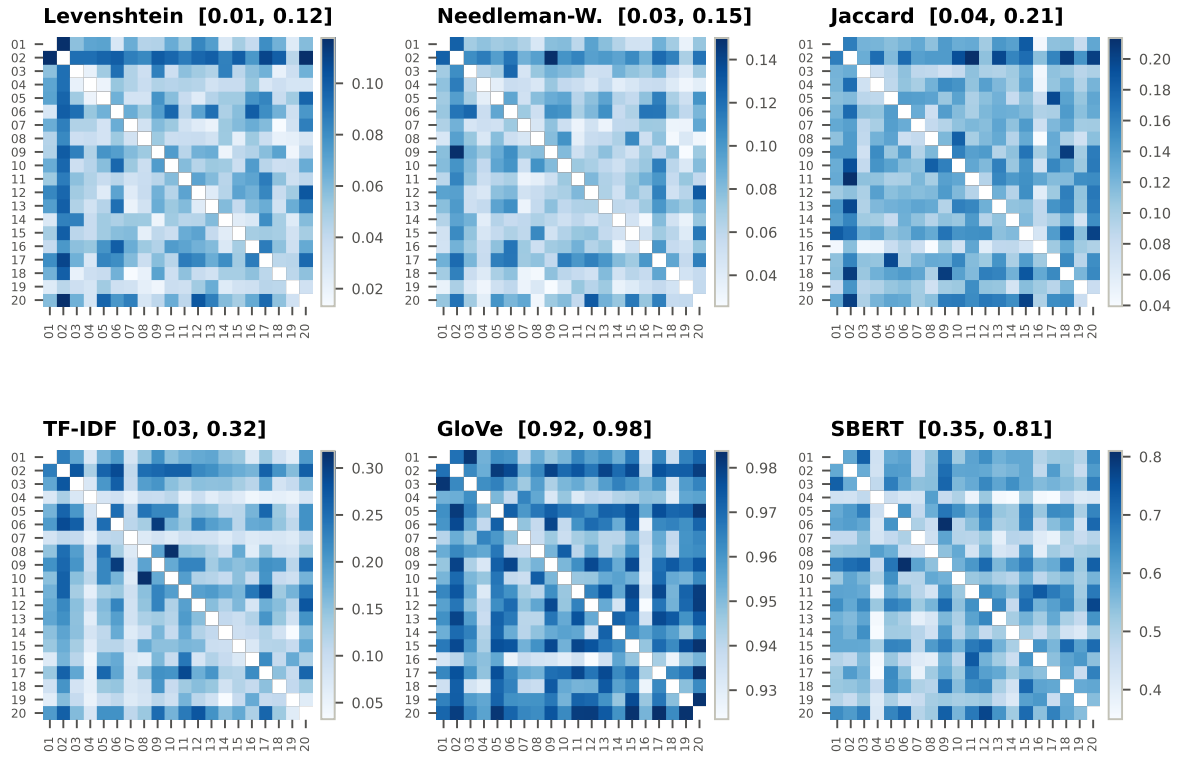


Figura 5: Matrices de similitud 20×20 de cada algoritmo; la escala de cada panel va del mínimo al máximo fuera de la diagonal.

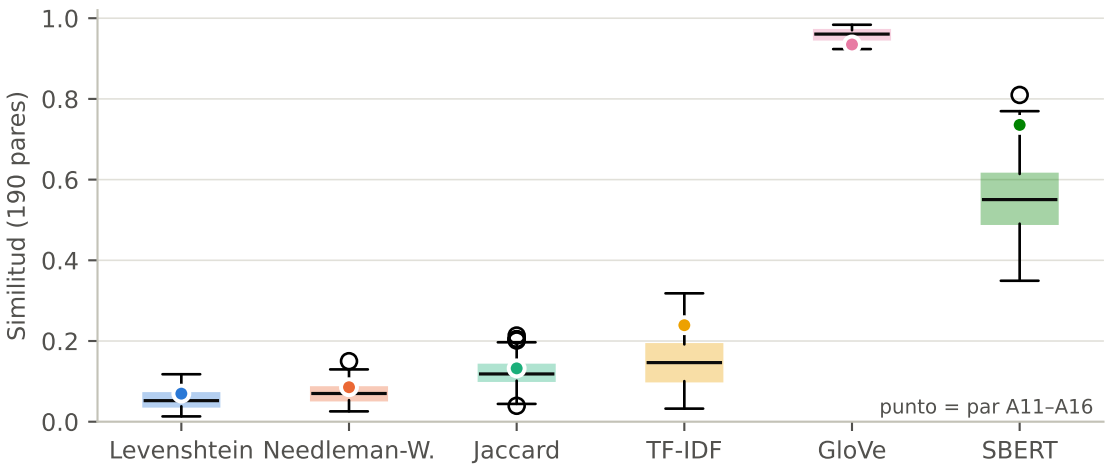


Figura 6: Distribución de las 190 similitudes por algoritmo; el punto marca el par del caso de estudio.

Tabla 10: Correlación de rangos de Spearman entre algoritmos sobre los 190 pares.

	Levenshtein	**Needleman-W.**	**Jaccard**	**TF-IDF**	**GloVe**	**SBERT**
Levenshtein	1.00	0.74	0.23	0.52	0.29	0.30
Needleman-W.	0.74	1.00	0.38	0.72	0.42	0.42
Jaccard	0.23	0.38	1.00	0.50	0.44	0.36
TF-IDF	0.52	0.72	0.50	1.00	0.55	0.69
GloVe	0.29	0.42	0.44	0.55	1.00	0.54
SBERT	0.30	0.42	0.36	0.69	0.54	1.00

Observaciones.

1. **Escala y discriminación.** Cada algoritmo usa una escala propia: un valor de 0.10 es alto para Levenshtein (media 0.056) y muy bajo para SBERT (media 0.554). Por eso no se deben comparar valores absolutos entre algoritmos sino posiciones relativas. Sentence-BERT tiene la mayor dispersión relativa (D.E. 0.089), es decir, la mejor capacidad para distinguir pares afines de pares lejanos.

2. **Anisotropía de GloVe.** El promedio de vectores GloVe da similitudes entre 0.92 y 0.98 (media 0.958, D.E. 0.014). Todos los abstracts comparten una gran componente común [14, 13]: palabras frecuentes del dominio y del registro académico que dominan el promedio. Como experimento adicional se restó a cada vector la media de los 20; la dispersión aumentó (media -0.049, D.E. 0.215), pero la concordancia con SBERT bajó de $\rho = 0.54$ a $\rho = 0.30$. El promedio de embeddings estáticos pierde el orden y el contexto, y resulta una representación débil para textos largos de un dominio estrecho.

3. **Concordancia entre enfoques.** SBERT concuerda más con TF-IDF ($\rho = 0.69$) que con Jaccard ($\rho = 0.36$) o con Levenshtein ($\rho = 0.30$): ponderar los términos por su rareza (idf) acerca el modelo vectorial clásico a la noción semántica. Levenshtein y Needleman-Wunsch concuerdan entre sí ($\rho = 0.74$) porque ambos miden alineamientos secuenciales.

4. **Costo.** Los algoritmos de programación dinámica son los más costosos ($\Theta(nm)$, con $n, m \approx 100$ raíces o $\approx 1\,400$ caracteres). Jaccard y TF-IDF son lineales. Los modelos de IA tienen un costo alto fuera de línea (la red neuronal) y casi nulo en línea (coseno sobre vectores precalculados).

Conclusión del Requerimiento 1

Para comparar abstracts, los algoritmos de edición (Levenshtein, Needleman-Wunsch) son útiles para detectar copias o versiones casi idénticas, pero no para medir afinidad temática. Jaccard y TF-IDF son una línea base léxica eficiente e interpretable; TF-IDF es el mejor de los clásicos. Sentence-BERT es el método más fiable para similitud semántica: discrimina mejor, no tiene palabras fuera de vocabulario y captura sinonimia y paráfrasis. GloVe con promedio, aunque es un modelo de IA, sufre anisotropía y desconoce términos recientes (*chatgpt*, *genai*).

6. Requerimiento 2: Agrupamiento jerárquico

6.1. Flujo general

El objetivo es construir un árbol (dendrograma) que represente la similitud entre los 20 abstracts y determinar qué criterio de agrupamiento produce los grupos más coherentes. El proceso tiene cinco etapas, implementadas en `bibliometria/clustering/`:

1. **Preprocesamiento** del abstract (sección 4).
2. **Vectorización**: TF-IDF con normalización L2 (representación clásica, la misma del Requerimiento 1). Como contraste se repite el análisis con los embeddings de Sentence-BERT.
3. **Matriz de distancias** $D \in \mathbb{R}^{20\times 20}$ con la distancia coseno $d(u,v) = 1 - \cos(u,v)$. Ward requiere distancia euclidiana; como los vectores tienen norma 1, $d_E(u,v) = \sqrt{2 - 2\cos(u,v)}$, que ordena los pares igual que la distancia coseno.
4. **Agrupamiento aglomerativo** con tres criterios exigidos (*single*, *complete* y *average linkage*) y Ward como cuarto criterio de comparación.
5. **Representación y evaluación**: dendrograma de cada criterio, correlación cofenética y coeficiente de silueta.

6.2. Algoritmo aglomerativo genérico

Los cuatro criterios comparten el mismo algoritmo (Johnson, 1967) [15] y sólo difieren en cómo se actualiza la distancia entre un clúster y el que resulta de fusionar otros dos. Esa actualización se expresa con la fórmula de Lance y Williams (1967) [16]:

$$d(h,\, i \cup j) = \alpha_i\, d(h,i) + \alpha_j\, d(h,j) + \beta\, d(i,j) + \gamma\, |d(h,i) - d(h,j)|. \tag{16}$$

Algoritmo 2 Agrupamiento jerárquico aglomerativo

Entrada: matriz de distancias D de $n \times n$, criterio de enlace
Salida: lista de $n-1$ fusiones $(i, j,$ altura, tamaño$)$

activos $\leftarrow \{0, \dots, n-1\}$, cada uno con tamaño 1
para $t \leftarrow 1$ **hasta** $n-1$ **hacer**
  $(i,j) \leftarrow \arg\min_{a<b\in \text{activos}} d(a,b)$ ▷ $O(k^2)$ con k clústeres activos
  crear el clúster $c = n + t - 1$ con tamaño $n_i + n_j$ y registrar la fusión a la altura $d(i,j)$
  para cada $h \in$ activos $\setminus \{i,j\}$ **hacer** ▷ $O(k)$
    $d(h,c) \leftarrow$ Lance-Williams$(d(h,i),\, d(h,j),\, d(i,j),\, n_h, n_i, n_j)$
  fin para
  activos $\leftarrow$ activos $\setminus \{i,j\} \cup \{c\}$
fin para

Complejidad. En la iteración con k clústeres activos, buscar el par mínimo cuesta $O(k^2)$ y actualizar las distancias $O(k)$, así que $T(n) = \sum_{k=2}^{n} O(k^2) = O(n^3)$ en tiempo y $O(n^2)$ en memoria. Para $n = 20$ se evalúan 1330 pares en total, un costo despreciable. Para corpus grandes existen variantes $O(n^2)$ [21, 20] (cadena de vecinos más cercanos, árbol de expansión mínima); aquí se prefirió la versión directa por su transparencia. La salida usa el formato de matriz de enlace de SciPy, lo que permitió verificar en las pruebas que las fusiones y alturas coinciden exactamente con `scipy.cluster.hierarchy.linkage`, sin usar SciPy en la aplicación.

6.3. Criterios de enlace

Tabla 11: Criterios de enlace: definición y coeficientes de Lance-Williams ($N = n_h + n_i + n_j$).

Criterio	Distancia entre clústeres A y B	Coeficientes
Single	$\min_{a\in A,\, b\in B} d(a,b)$	$\alpha_i = \alpha_j = \frac{1}{2},\ \beta = 0,\ \gamma = -\frac{1}{2}$
Complete	$\max_{a\in A,\, b\in B} d(a,b)$	$\alpha_i = \alpha_j = \frac{1}{2},\ \beta = 0,\ \gamma = +\frac{1}{2}$
Average (UPGMA)	$\frac{1}{\lvert A\rvert\lvert B\rvert}\sum_{a\in A}\sum_{b\in B} d(a,b)$	$\alpha_i = \frac{n_i}{n_i+n_j},\ \alpha_j = \frac{n_j}{n_i+n_j},\ \beta = \gamma = 0$
Ward	$\frac{\lvert A\rvert\lvert B\rvert}{\lvert A\rvert+\lvert B\rvert}\lVert\vec{c}_A - \vec{c}_B\rVert^2$	$\alpha_i = \frac{n_h+n_i}{N},\ \alpha_j = \frac{n_h+n_j}{N},\ \beta = -\frac{n_h}{N},\ \gamma = 0$

- **Single linkage** une los clústeres por sus elementos más cercanos. Detecta formas alargadas, pero sufre el *efecto cadena*: un elemento intermedio puede unir grupos distintos.
- **Complete linkage** usa los elementos más lejanos y produce grupos compactos de diámetro acotado; es sensible a los valores atípicos.
- **Average linkage** promedia todas las distancias entre pares, un punto intermedio entre los dos anteriores.
- **Ward** [17] fusiona el par que menos aumenta la varianza interna (suma de cuadrados) y tiende a formar grupos de tamaño parecido. En la implementación, la recurrencia se aplica sobre distancias euclidianas: $d(h, i\cup j) = \sqrt{((n_h+n_i)d_{hi}^2 + (n_h+n_j)d_{hj}^2 - n_h d_{ij}^2)/N}$.

Los cuatro criterios son *monótonos*: las alturas de fusión nunca disminuyen, de modo que el dendrograma no presenta inversiones.

`clustering/agglomerative.py — fórmula de Lance-Williams`

```python
def lance_williams(method: str, d_hi: float, d_hj: float, d_ij: float, n_h: int, n_i: int, n_j: int)
    -> float:
    if method == "single":
        return d_hi if d_hi < d_hj else d_hj
    if method == "complete":
        return d_hi if d_hi > d_hj else d_hj
    if method == "average":
        return (n_i * d_hi + n_j * d_hj) / (n_i + n_j)
    if method == "ward":
        total = n_h + n_i + n_j
        value = ((n_h + n_i) * d_hi * d_hi + (n_h + n_j) * d_hj * d_hj - n_h * d_ij * d_ij) / total
        return math.sqrt(value if value > 0 else 0.0)
    raise ValueError(f"Criterio de enlace desconocido: {method}")
```

6.4. Ejemplo paso a paso

Para mostrar el funcionamiento interno se aplican los tres criterios a un subconjunto de 5 artículos sobre IA generativa, con sus distancias coseno sobre TF-IDF (tabla 12).

Tabla 12: Distancias coseno entre cinco abstracts (TF-IDF).

	A11	A15	A16	A18	A19
A11	0.0000	0.8358	0.7608	0.8626	0.8324
A15	0.8358	0.0000	0.9255	0.8798	0.8982
A16	0.7608	0.9255	0.0000	0.8952	0.8212
A18	0.8626	0.8798	0.8952	0.0000	0.8939
A19	0.8324	0.8982	0.8212	0.8939	0.0000

Iteración 1 (común a los tres criterios): la menor distancia es $d(A11, A16) = 0.7608$, así que se fusionan en $C_1 = \{A11, A16\}$. La distancia de $A15$ al nuevo clúster depende del criterio:

$$\begin{aligned} \text{single:} &\quad \min\{d(A15, A11),\, d(A15, A16)\} = \min\{0.8358,\, 0.9255\} = 0.8358 \\ \text{complete:} &\quad \max\{0.8358,\, 0.9255\} = 0.9255 \\ \text{average:} &\quad \tfrac{1}{2}\,0.8358 + \tfrac{1}{2}\,0.9255 = 0.8807 \end{aligned}$$

Las tablas 13–15 muestran todas las iteraciones; la última columna da las distancias actualizadas al clúster recién creado.

Tabla 13: Iteraciones con *single linkage*.

t	**Clúster creado**	**Altura**	**Distancias actualizadas**
1	{A11, A16}	0.7608	d({A15}) = 0.8358; d({A18}) = 0.8626; d({A19}) = 0.8212
2	{A19, A11, A16}	0.8212	d({A15}) = 0.8358; d({A18}) = 0.8626
3	{A15, A19, A11, A16}	0.8358	d({A18}) = 0.8626
4	{A18, A15, A19, A11, A16}	0.8626	—

Tabla 14: Iteraciones con *complete linkage*.

t	**Clúster creado**	**Altura**	**Distancias actualizadas**
1	{A11, A16}	0.7608	d({A15}) = 0.9255; d({A18}) = 0.8952; d({A19}) = 0.8324
2	{A19, A11, A16}	0.8324	d({A15}) = 0.9255; d({A18}) = 0.8952
3	{A15, A18}	0.8798	d({A19, A11, A16}) = 0.9255
4	{A19, A11, A16, A15, A18}	0.9255	—

Tabla 15: Iteraciones con *average linkage*.

t	**Clúster creado**	**Altura**	**Distancias actualizadas**
1	{A11, A16}	0.7608	d({A15}) = 0.8807; d({A18}) = 0.8789; d({A19}) = 0.8268
2	{A19, A11, A16}	0.8268	d({A15}) = 0.8865; d({A18}) = 0.8839
3	{A15, A18}	0.8798	d({A19, A11, A16}) = 0.8852
4	{A19, A11, A16, A15, A18}	0.8852	—

El ejemplo muestra las diferencias entre criterios. Con *single*, A19, A15 y A18 se van uniendo de uno en uno al mismo clúster, que crece en cadena. Con *complete*, A15 y A18 forman primero su propio grupo, porque su distancia al bloque {A19, A11, A16} se mide por el par más lejano (0.9255). *Average* produce el mismo orden de fusiones que *complete*, pero con alturas intermedias.

6.5. Métricas de evaluación interna

Correlación cofenética (CPCC). La distancia cofenética c_{ij} es la altura a la que i y j quedan por primera vez en el mismo clúster. El CPCC (Sokal y Rohlf, 1962) [18] es la correlación de Pearson entre las distancias originales y las cofenéticas:

$$\mathrm{CPCC} = \frac{\sum_{i<j}(d_{ij} - \bar{d})(c_{ij} - \bar{c})}{\sqrt{\sum_{i<j}(d_{ij} - \bar{d})^2 \; \sum_{i<j}(c_{ij} - \bar{c})^2}} \in [-1, 1]. \tag{17}$$

Mide qué tan fielmente el árbol conserva las distancias reales.

Coeficiente de silueta. Al cortar el árbol en k grupos, para cada documento i se calculan $a(i)$, la distancia media a los demás miembros de su grupo, y $b(i)$, la menor distancia media a los miembros de otro grupo (Rousseeuw, 1987) [19]:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}, \qquad S_k = \frac{1}{n}\sum_{i=1}^{n} s(i) \in [-1, 1], \tag{18}$$

con $s(i) = 0$ si el grupo de i es unitario. Valores cercanos a 1 indican grupos compactos y separados; cercanos a 0, grupos que se solapan.

Criterio de decisión. Se considera más coherente el criterio con mayor **silueta media** para $k \in [2, 8]$, porque es robusta frente a la elección de un único k. El CPCC se usa como desempate. Para comparar criterios con una misma vara de medida, la silueta de todos se calcula con la distancia coseno. Ambas métricas están implementadas desde cero en `evaluation.py`, y las pruebas confirman que coinciden con `scipy.cluster.hierarchy.cophenet` y `sklearn.metrics.silhouette_score` con una precisión de 10^{-12}.

clustering/evaluation.py — coeficiente de silueta

```python
def silhouette_samples(D: list[list[float]], labels: list[int]) -> list[float]:
    n = len(D)
    groups: dict[int, list[int]] = {}
    for i, g in enumerate(labels):
        groups.setdefault(g, []).append(i)
    if len(groups) < 2 or len(groups) > n - 1:
        raise ValueError("La silueta requiere 2 <= k <= n-1 grupos")
    scores: list[float] = []
    for i in range(n):
        own = groups[labels[i]]
        if len(own) == 1:
            scores.append(0.0)
            continue
        a = sum(D[i][j] for j in own if j != i) / (len(own) - 1)
        b = math.inf
        for g, members in groups.items():
            if g == labels[i]:
                continue
            avg = sum(D[i][j] for j in members) / len(members)
            if avg < b:
                b = avg
        denom = a if a > b else b
        scores.append(0.0 if denom == 0 else (b - a) / denom)
    return scores
```

6.6. Resultados

Tabla 16: CPCC y silueta para $k = 2, \dots, 8$. En negrilla, el criterio más coherente entre los tres exigidos.

Criterio	CPCC	S_2	S_3	S_4	S_5	S_6	S_7	S_8	Media	k^*
Representación TF-IDF										
Single	0.764	0.074	0.071	0.056	0.033	0.025	0.019	0.010	0.041	2
Complete	0.587	0.036	0.038	0.038	0.023	0.024	0.025	0.025	0.030	3
Average (UPGMA)	0.816	0.083	0.071	0.056	0.035	0.026	0.019	0.026	**0.045**	2
Ward (adicional)	0.538	0.033	0.034	0.034	0.039	0.031	0.032	0.029	0.033	5
Representación Sentence-BERT										
Single	0.770	0.159	0.152	0.105	0.077	0.069	0.052	0.023	0.091	2
Complete	0.444	0.043	0.071	0.091	0.078	0.071	0.091	0.083	0.076	7
Average (UPGMA)	0.798	0.201	0.175	0.130	0.090	0.068	0.037	0.071	**0.110**	2
Ward (adicional)	0.603	0.186	0.108	0.126	0.113	0.096	0.103	0.116	0.121	2

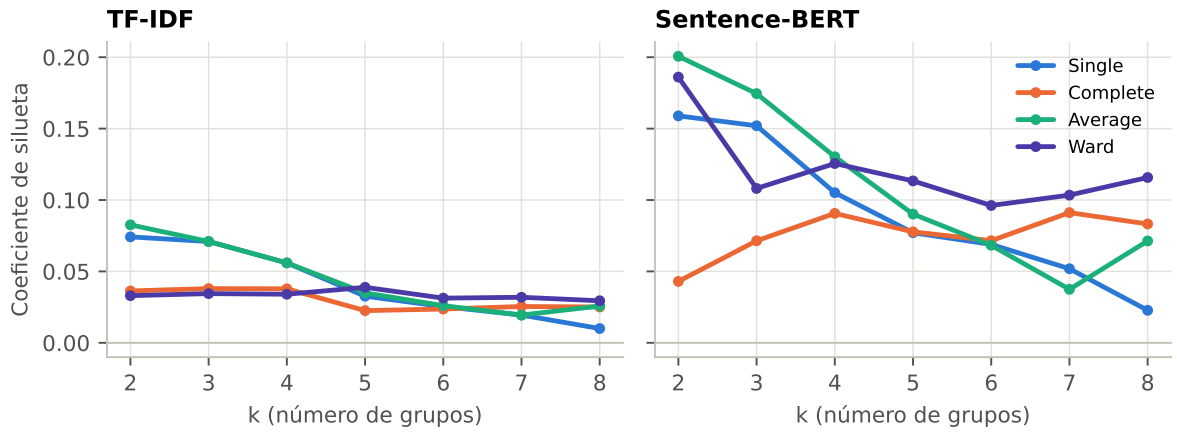


Figura 7: Coeficiente de silueta según el número de grupos k.

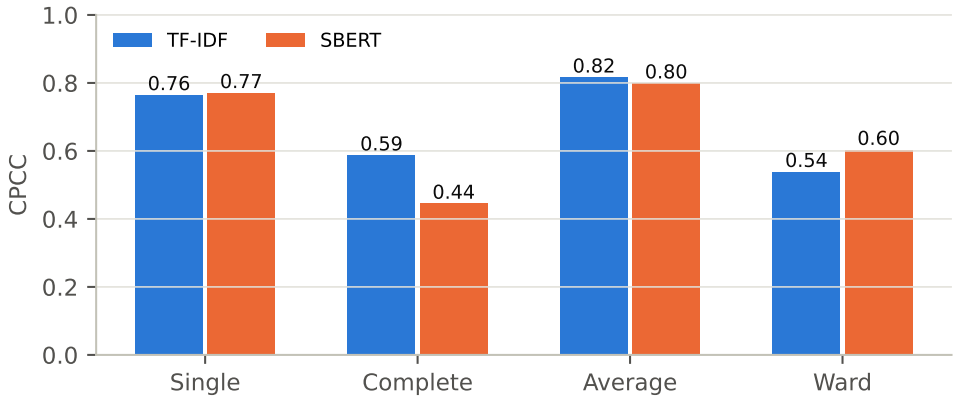


Figura 8: Correlación cofenética por criterio y representación.

Dendrogramas

Los dendrogramas se dibujan con código propio (`dendrogram.py`). El orden de las hojas se obtiene con un recorrido en profundidad desde la raíz; cada hoja se ubica en $x = 5 + 10i$ y cada fusión, en el punto medio de sus hijos y a la altura de la distancia de fusión. Las pruebas comprueban que esta geometría coincide con la de `scipy.cluster.hierarchy.dendrogram`. Las hojas se rotulan con el código del artículo, el primer autor y el año, un cruce con los metadatos extraídos. Las ramas se colorean según el corte en $k = 4$ grupos (línea roja).

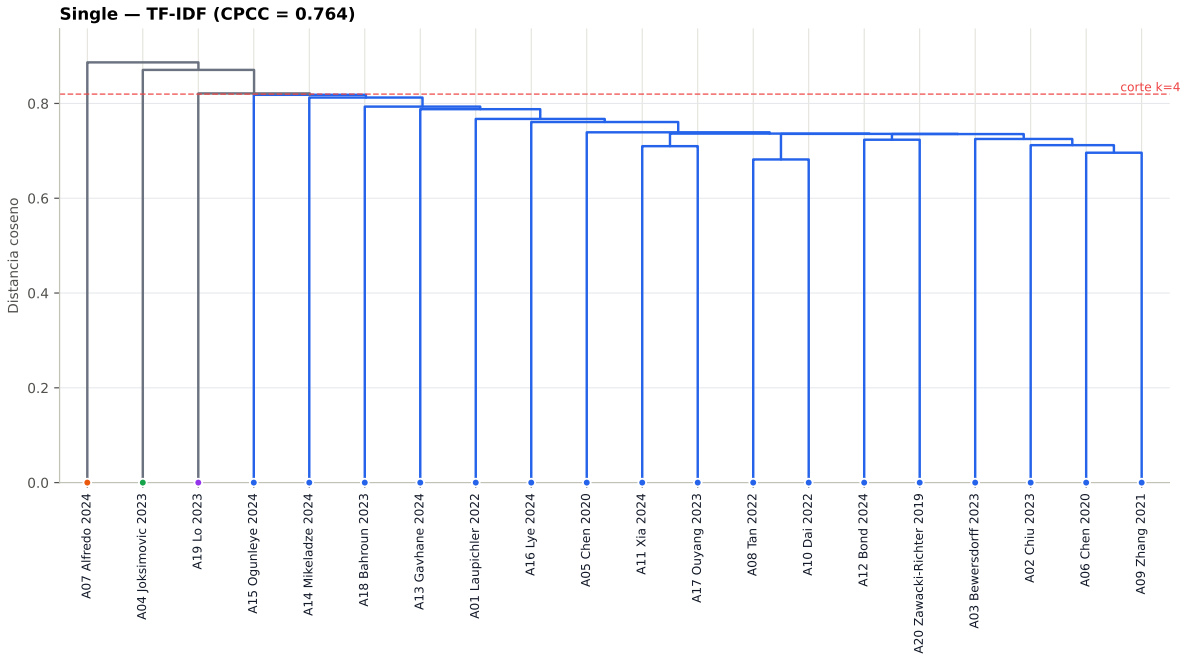


Figura 9: Dendrograma *single linkage* (TF-IDF, distancia coseno).

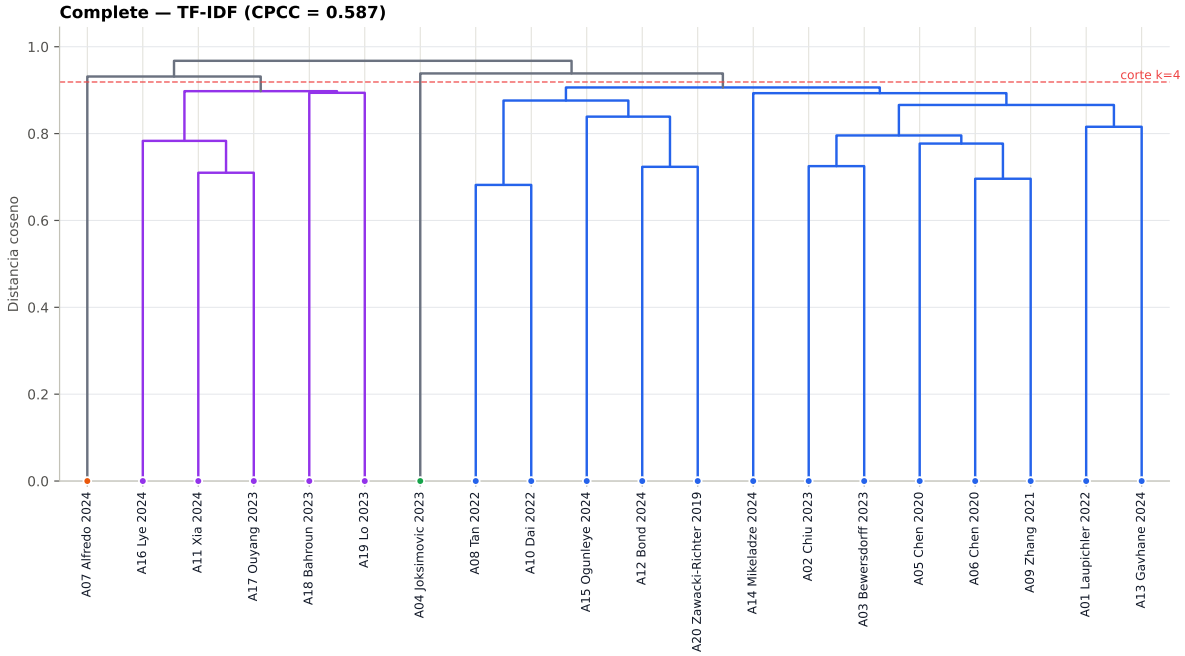


Figura 10: Dendrograma *complete linkage* (TF-IDF, distancia coseno).

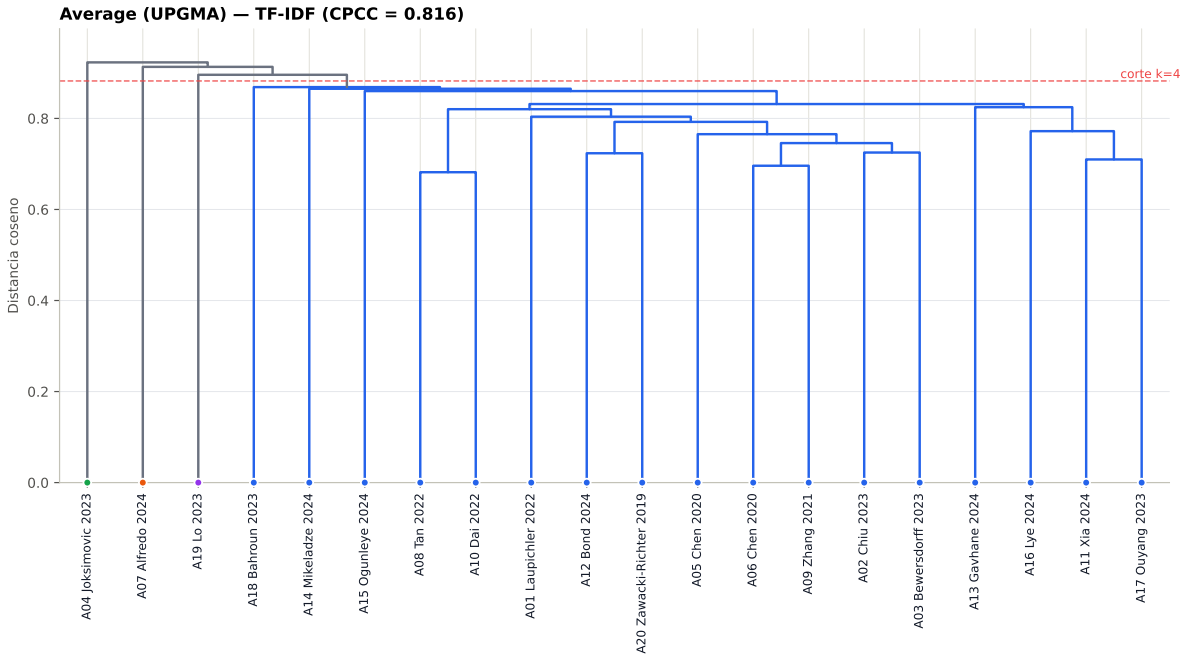


Figura 11: Dendrograma *average linkage* (TF-IDF, distancia coseno).

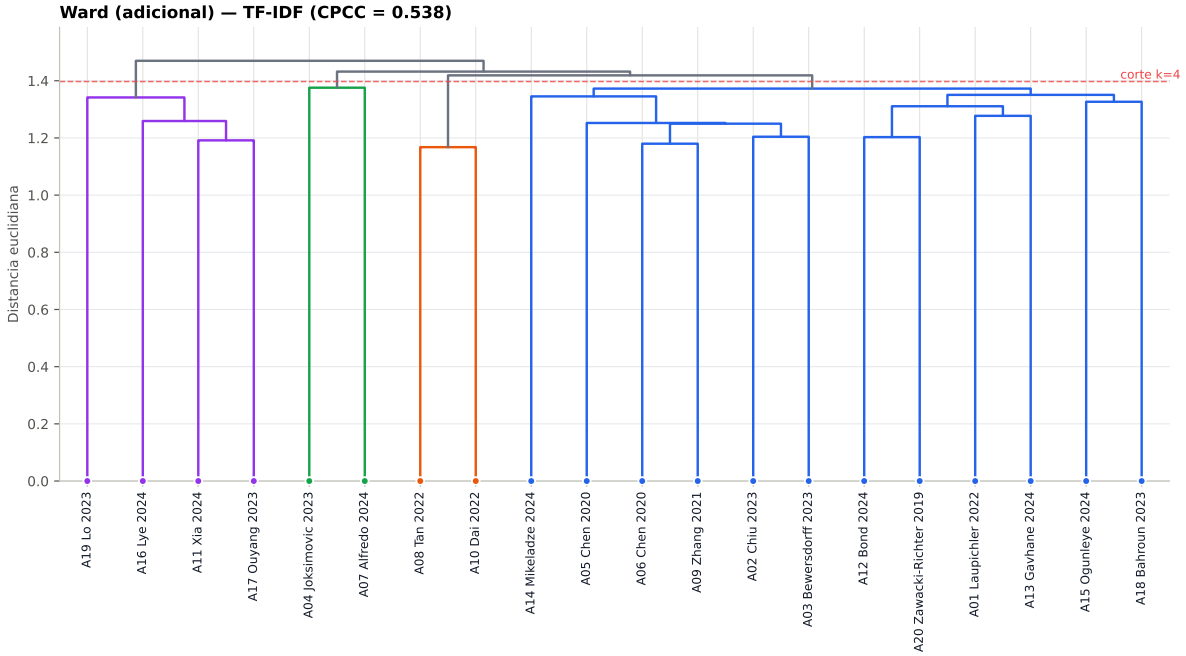


Figura 12: Dendrograma de Ward (TF-IDF, distancia euclidiana), criterio adicional.

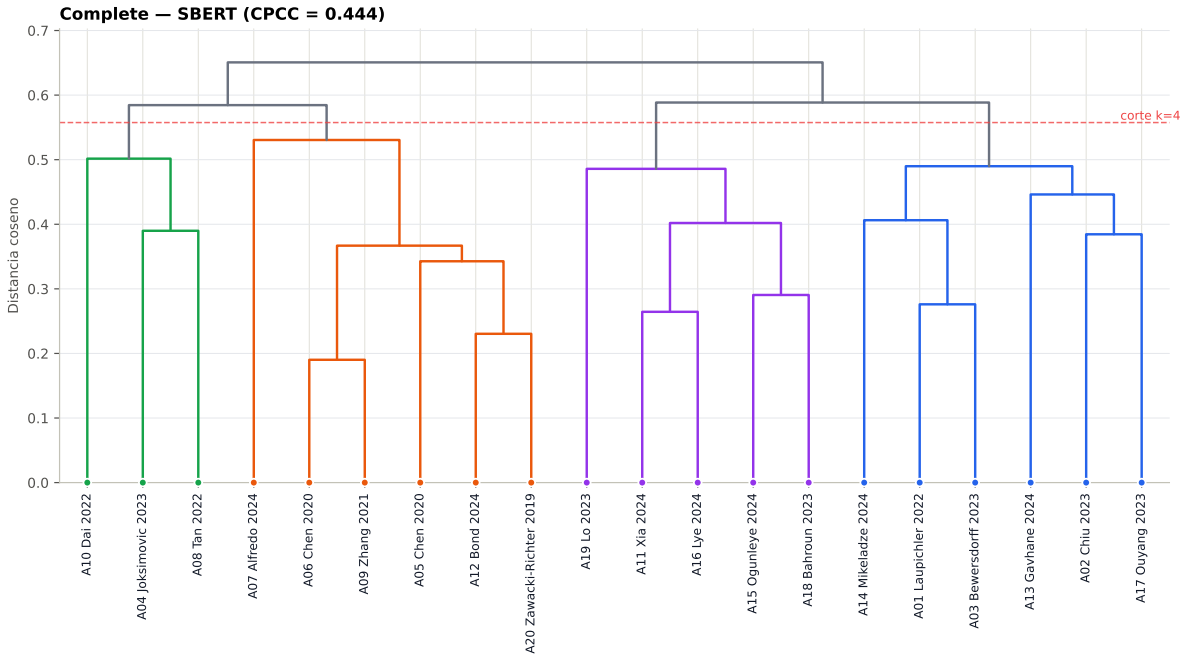


Figura 13: Dendrograma *complete linkage* sobre los embeddings de Sentence-BERT.

Interpretación

- **Average linkage es el criterio más coherente** entre los tres exigidos según las métricas internas, en las dos representaciones: obtiene el mayor CPCC (0.816 con TF-IDF y 0.798 con SBERT) y la mayor silueta media entre *single*, *complete* y *average* (0.045 y 0.110). Su árbol es el que mejor conserva las distancias originales.
- **Single** queda cerca en CPCC (0.764) pero muestra el efecto cadena: los artículos se unen de uno en uno a un bloque principal. **Complete** tiene el CPCC más bajo de los tres exigidos (0.587) y la menor silueta media, aunque forma los grupos más compactos.
- **Los valores de silueta son bajos** (por debajo de 0.21 en todos los casos). Esto no indica un error de implementación: los 20 artículos son revisiones sobre IA en educación y comparten buena parte de su vocabulario, así que no hay grupos naturales bien separados.
- **Valores atípicos.** Con TF-IDF, *average* y *single* alcanzan su mejor silueta en $k = 2$ separando un único artículo, y en $k = 4$ aíslan a A04, A07 y A19 (resolución de problemas complejos, analítica de aprendizaje centrada en humanos y la revisión rápida sobre ChatGPT), que usan un vocabulario muy particular:
  - **Grupo 1** (17): A01, A02, A03, A05, A06, A08, A09, A10, A11, A12, A13, A14, A15, A16, A17, A18, A20.
  - **Grupo 2** (1): A04.
  - **Grupo 3** (1): A07.
  - **Grupo 4** (1): A19.
- **La representación importa más que el criterio.** Con los embeddings de Sentence-BERT la silueta media de todos los criterios aumenta entre 2 y 4 veces, y los grupos son temáticamente nítidos. Por ejemplo, *complete* con $k = 4$ produce:
  - **Grupo 1** (6): A01, A02, A03, A13, A14, A17.
  - **Grupo 2** (3): A04, A08, A10.
  - **Grupo 3** (6): A05, A06, A07, A09, A12, A20.
  - **Grupo 4** (5): A11, A15, A16, A18, A19.

Son, en orden: alfabetización y competencias en IA y evaluación; IA para resolución de problemas, colaboración y simulación; mapeos y metarrevisiones del campo AIEd; e IA generativa y ChatGPT.

Conclusión del Requerimiento 2

Según la métrica de evaluación interna (silueta media, con el CPCC como desempate), **average linkage** produce los agrupamientos más coherentes entre los tres criterios exigidos, con las dos representaciones. Es además el árbol que mejor preserva las distancias (CPCC ≈ 0.8). Como matiz práctico, *complete* produce particiones más balanceadas, y la representación semántica (SBERT) mejora la separación de todos los criterios. Ward, evaluado como complemento, obtiene la mejor silueta media con SBERT (0.121).

7. Requerimiento 3: Búsqueda de patrones (coincidencia exacta)

7.1. Flujo general

El usuario ingresa un patrón P (longitud m) y la aplicación localiza todas sus ocurrencias exactas en los abstracts T (longitud n) de los 20 artículos. Como cruce de información, la búsqueda puede extenderse a los títulos, los autores y las palabras clave; los resultados se muestran con su contexto, el título, los autores y el año del artículo. Los tres algoritmos (`bibliometria/string_matching/`) se implementaron desde cero: **no se usa `find()`, `index()`, `in`, `count()` ni expresiones regulares**, y toda verificación se hace comparando carácter por carácter, lo que permite contar las comparaciones. Por defecto la búsqueda no distingue mayúsculas: texto y patrón se pasan a minúsculas carácter por carácter, conservando la longitud, de modo que las posiciones no cambian.

Cada ejecución devuelve las posiciones, el número de comparaciones, los tiempos de preprocesamiento y de búsqueda (medidos por separado con `perf_counter_ns`) y, si se solicita, una traza con cada comparación y cada desplazamiento, que la interfaz reproduce paso a paso.

7.2. Caso de estudio

Se eligió el patrón **`"assessment"`** ($m = 10$) y el párrafo formado por las dos primeras oraciones del abstract de A16 ($n = 267$ caracteres):

Texto del caso de estudio (A16)

The emergence of generative artificial intelligence (AI) such as ChatGPT has sparked significant assessment concerns within tertiary education. Assessment concerns have largely revolved around academic integrity issues among students, such as plagiarism and cheating.

El patrón aparece en las posiciones **97, 144**; la segunda ocurrencia está escrita «Assessment» y se encuentra gracias a la normalización a minúsculas. Con la fuerza bruta como referencia (300 comparaciones), los tres algoritmos se comportan así:

Tabla 17: Resultados del caso de estudio ($n = 267$, $m = 10$). Rabin-Karp con $q = 13$.

Algoritmo	Comparaciones	Alineaciones	Preproc. (µs)	Búsqueda (µs)
Fuerza bruta	300	258	—	66.2
KMP	289	247	4.2	93.0
Boyer-Moore	54	31	5.6	23.2
Rabin-Karp ($q = 13$)	39	258	5.0	92.7

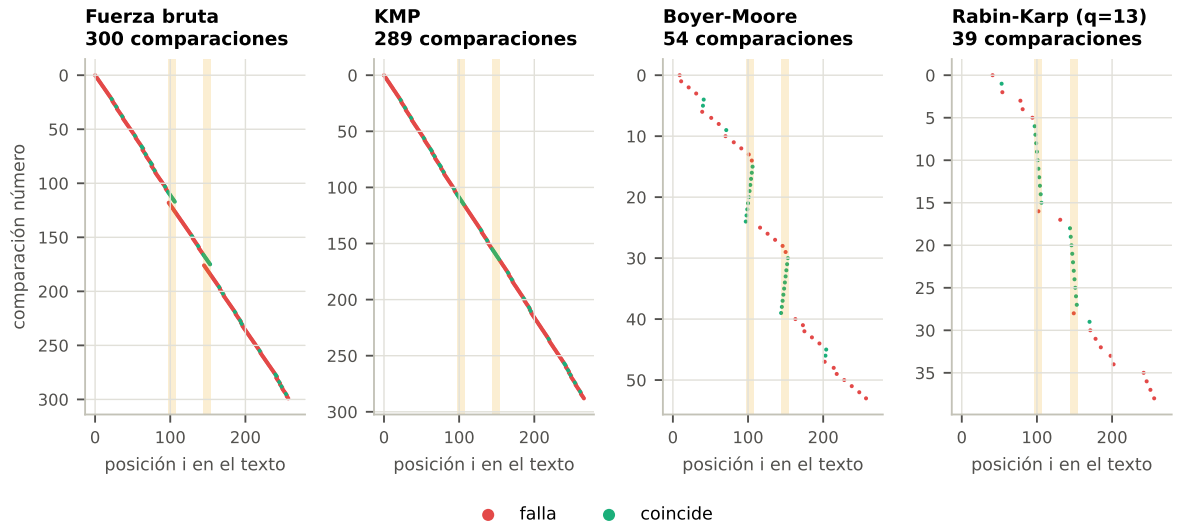


Figura 14: Trayectoria de las comparaciones de cada algoritmo sobre el párrafo: posición del texto comparada (eje x) en cada paso (eje y, hacia abajo). Las franjas amarillas son las ocurrencias.

La figura 14 resume el comportamiento de cada uno. La fuerza bruta y KMP recorren el texto de forma densa, pero KMP nunca retrocede: su trayectoria es monótona. Boyer-Moore avanza a saltos de hasta m posiciones y compara pocos caracteres. Rabin-Karp sólo compara caracteres en las ventanas cuyo *hash* coincide con el del patrón.

7.3. Algoritmo de Knuth-Morris-Pratt

Idea. Cuando ocurre una discrepancia después de emparejar q caracteres, esos q caracteres del texto ya son conocidos: coinciden con $P[0..q-1]$. KMP [22, 26] aprovecha esa información para desplazar el patrón sin retroceder sobre el texto, usando la **función de prefijos**:

$$\pi[q] = \max\{\, k < q+1 \;:\; P[0..k-1] \text{ es prefijo propio y sufijo de } P[0..q] \,\}. \tag{19}$$

Tras una discrepancia con q caracteres emparejados, la búsqueda continúa con $q \leftarrow \pi[q-1]$.

Algoritmo 3 Función de prefijos y búsqueda KMP

```
1:  π[0] ← 0; k ← 0
2:  para q ← 1 hasta m − 1 hacer                          ▷ preprocesamiento Θ(m)
3:      mientras k > 0 y P[k] ≠ P[q] hacer k ← π[k − 1]
4:      fin mientras
5:      si P[k] = P[q] entonces k ← k + 1
6:      fin si
7:      π[q] ← k
8:  fin para
9:  q ← 0
10: para i ← 0 hasta n − 1 hacer                          ▷ búsqueda Θ(n)
11:     mientras q > 0 y P[q] ≠ T[i] hacer q ← π[q − 1]
12:     fin mientras
13:     si P[q] = T[i] entonces q ← q + 1
14:     fin si
15:     si q = m entonces reportar i − m + 1; q ← π[m − 1]
16:     fin si
17: fin para
```

Tabla 18: Función de prefijos de `"assessment"`.

q	0	1	2	3	4	5	6	7	8	9
$P[q]$	a	s	s	e	s	s	m	e	n	t
$\pi[q]$	0	0	0	0	0	0	0	0	0	0

Tabla π del caso de estudio. Todos los valores son 0: ningún prefijo propio de `assessment` es también sufijo de un prefijo (la `a` inicial no se repite). Esto es habitual en palabras del lenguaje natural, y por eso la ventaja de KMP sobre la fuerza bruta es pequeña en este tipo de texto (289 frente a 300 comparaciones). La ventaja crece con patrones repetitivos, como secuencias de ADN o el peor caso de la sección 7.6.

Para ilustrar el cálculo de π con bordes no triviales se usa la palabra del corpus `"interdisciplinary"`, cuyo prefijo `in` reaparece en `discipl`**`in`**`ary`:

Tabla 19: Función de prefijos de `"interdisciplinary"`.

q	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
$P[q]$	i	n	t	e	r	d	i	s	c	i	p	l	i	n	a	r	y
$\pi[q]$	0	0	0	0	0	0	**1**	0	0	**1**	0	0	**1**	**2**	0	0	0

- $q = 6$ (i): $k = 0$; $P[0] = P[6] \Rightarrow \pi[6] = 1$.
- $q = 7$ (s): $k = 1 \to 0$; $P[0] \neq P[7] \Rightarrow \pi[7] = 0$.
- $q = 9$ (i): $k = 0$; $P[0] = P[9] \Rightarrow \pi[9] = 1$.
- $q = 10$ (p): $k = 1 \to 0$; $P[0] \neq P[10] \Rightarrow \pi[10] = 0$.
- $q = 12$ (i): $k = 0$; $P[0] = P[12] \Rightarrow \pi[12] = 1$.
- $q = 13$ (n): $k = 1$; $P[1] = P[13] \Rightarrow \pi[13] = 2$.
- $q = 14$ (a): $k = 2 \to 0$; $P[0] \neq P[14] \Rightarrow \pi[14] = 0$.

Traza de la búsqueda. La tabla 20 muestra los primeros eventos y los que rodean la primera ocurrencia. Cuando $q = 0$ y hay discrepancia, el patrón avanza una posición; al encontrar la ocurrencia, la búsqueda continúa con $q = \pi[m - 1] = 0$.

Tabla 20: Fragmento de la traza de KMP (i: posición en el texto, q: caracteres emparejados, $s = i - q$: alineación).

Evento	Índices	Caracteres	Resultado	Alineación
compara	$i = 0, q = 0$	t vs a	falla	$s = 0$
desplaza			π no aplica (q = 0): avanzar 1	$s : 0 \to 1$
compara	$i = 1, q = 0$	h vs a	falla	$s = 1$
desplaza			π no aplica (q = 0): avanzar 1	$s : 1 \to 2$
compara	$i = 2, q = 0$	e vs a	falla	$s = 2$
desplaza			π no aplica (q = 0): avanzar 1	$s : 2 \to 3$
compara	$i = 3, q = 0$	␣ vs a	falla	$s = 3$
desplaza			π no aplica (q = 0): avanzar 1	$s : 3 \to 4$
			⋮	
compara	$i = 95, q = 0$	t vs a	falla	$s = 95$
desplaza			π no aplica (q = 0): avanzar 1	$s : 95 \to 96$
compara	$i = 96, q = 0$	␣ vs a	falla	$s = 96$
desplaza			π no aplica (q = 0): avanzar 1	$s : 96 \to 97$
compara	$i = 97, q = 0$	a vs a	coincide	$s = 97$
compara	$i = 98, q = 1$	s vs s	coincide	$s = 97$
compara	$i = 99, q = 2$	s vs s	coincide	$s = 97$
compara	$i = 100, q = 3$	e vs e	coincide	$s = 97$
compara	$i = 101, q = 4$	s vs s	coincide	$s = 97$
compara	$i = 102, q = 5$	s vs s	coincide	$s = 97$
compara	$i = 103, q = 6$	m vs m	coincide	$s = 97$
compara	$i = 104, q = 7$	e vs e	coincide	$s = 97$
compara	$i = 105, q = 8$	n vs n	coincide	$s = 97$
compara	$i = 106, q = 9$	t vs t	coincide	$s = 97$
ocurrencia				$s = 97$
desplaza			ocurrencia: continuar con $\pi[9] = 0$	$s : 97 \to 107$
compara	$i = 107, q = 0$	␣ vs a	falla	$s = 107$

Complejidad. El índice i nunca retrocede, y q aumenta como máximo una vez por cada i; cada iteración del *while* disminuye q. Por análisis amortizado, el total de comparaciones de la búsqueda es a lo sumo $2n$, y el del preprocesamiento a lo sumo $2m$: $\Theta(m)$ y $\Theta(n)$, con memoria $O(m)$ para π.

string_matching/kmp.py — búsqueda (sin las líneas de traza)

```python
def kmp_search(text: str, pattern: str, trace: bool = False, max_trace: int = DEFAULT_MAX_TRACE) ->
     MatchResult:
    validate(text, pattern)
    n, m = len(text), len(pattern)
    tracer = Tracer(trace, max_trace)
    build_steps: list[dict] | None = [] if trace else None

    t0 = time.perf_counter_ns()
    pi = prefix_function(pattern, build_steps)
    t1 = time.perf_counter_ns()

    positions: list[int] = []
    comparisons = 0
    shifts = 0
    q = 0  # número de caracteres del patrón emparejados
    for i in range(n):
        while True:
            comparisons += 1
            if pattern[q] == text[i]:
                q += 1
```

```
                break
            if q == 0:
                shifts += 1
                break
            shifts += 1
            q = pi[q - 1]
        if q == m:
            positions.append(i - m + 1)
            if m > 0:
                shifts += 1
            q = pi[q - 1]
    t2 = time.perf_counter_ns()

    return MatchResult(
        algorithm="kmp",
        pattern=pattern,
        text_length=n,
        positions=positions,
        comparisons=comparisons,
        preprocessing_ns=t1 - t0,
        search_ns=t2 - t1,
        alignments=shifts + 1 if n >= m else 0,
        tables={"pi": pi, "pi_steps": build_steps or []},
        trace=tracer.events,
        trace_truncated=tracer.truncated,
    )
```

7.4. Algoritmo de Boyer-Moore

Idea. Boyer y Moore (1977) [23] comparan el patrón **de derecha a izquierda**. Ante una discrepancia en la posición j del patrón, con el carácter $c = T[s+j]$ del texto, aplican dos heurísticas y desplazan el patrón según la mayor:

- **Carácter malo:** alinear c con su última aparición en el patrón: $\delta_{bc} = j - \text{última}(c)$, con $\text{última}(c) = -1$ si c no aparece en P.
- **Sufijo bueno:** el sufijo $P[j+1..m-1]$ ya coincidió. Se desplaza hasta su siguiente aparición en el patrón precedida por un carácter distinto de $P[j]$ (sufijo bueno fuerte) o, si no existe, hasta el prefijo más largo de P que sea sufijo de lo emparejado: $\delta_{gs} = gs[j+1]$.

$$s \leftarrow s + \max(\delta_{bc},\, \delta_{gs}); \qquad \text{tras una ocurrencia: } s \leftarrow s + gs[0]. \tag{20}$$

Tabla 21: Tabla del carácter malo de `"assessment"`.

Carácter c	a	s	e	m	n	t	otro
$\text{última}(c)$	0	5	7	6	8	9	-1

Tablas heurísticas del caso de estudio. La tabla se llena recorriendo el patrón de izquierda a derecha y sobrescribiendo la posición de cada carácter; por eso `s` queda en 5 y `e` en 7, su última aparición.

Tabla 22: Tabla del sufijo bueno gs de `"assessment"` (falla en $P[j] \Rightarrow$ se usa $gs[j+1]$).

j	0	1	2	3	4	5	6	7	8	9	10
$P[j]$	a	s	s	e	s	s	m	e	n	t	–
$gs[j]$	10	10	10	10	10	10	10	10	10	10	1

La tabla se construye con el arreglo de bordes `bpos` [27] en dos fases, cada una $O(m)$. $gs[10] = 1$ corresponde a una falla en el último carácter, cuando todavía no se ha emparejado ningún sufijo. Todos los demás valores son 10: ningún sufijo de `assessment` (`t`, `nt`, `ent`, …) reaparece en el patrón, y ningún prefijo coincide con un sufijo, así que el sufijo bueno permite saltar el patrón completo.

Iteraciones y desplazamientos sobre el texto. La tabla 23 grafica las 31 alineaciones. En cada fila se ve la ventana del texto alrededor de la alineación s: en verde los caracteres que coincidieron, en rojo la discrepancia y en gris claro las posiciones de la ventana que no fue necesario comparar.

Tabla 23: Iteraciones de Boyer-Moore sobre el texto del caso de estudio.

k	s	**Ventana** $T[s-2..s+m+1]$	**Comp.**	**Regla aplicada**	**Despl.**
1	0	`the␣emergenc`	1	`e`: $\delta_{bc}=2$, $\delta_{gs}=1$	2
2	2	`the␣emergence␣`	1	`c`: $\delta_{bc}=10$, $\delta_{gs}=1$	10
3	12	`nce␣of␣generat`	1	`r`: $\delta_{bc}=10$, $\delta_{gs}=1$	10
4	22	`erative␣artifi`	1	`i`: $\delta_{bc}=10$, $\delta_{gs}=1$	10
5	32	`tificial␣intel`	3	`i`: $\delta_{bc}=8$, $\delta_{gs}=10$	10
6	42	`ntelligence␣(a`	1	`␣`: $\delta_{bc}=10$, $\delta_{gs}=1$	10
7	52	`e␣(ai)␣such␣as`	1	`␣`: $\delta_{bc}=10$, $\delta_{gs}=1$	10
8	62	`h␣as␣chatgpt␣h`	2	`p`: $\delta_{bc}=9$, $\delta_{gs}=10$	10
9	72	`pt␣has␣sparked`	1	`k`: $\delta_{bc}=10$, $\delta_{gs}=1$	10
10	82	`rked␣significa`	1	`i`: $\delta_{bc}=10$, $\delta_{gs}=1$	10
11	92	`ficant␣assessm`	1	`s`: $\delta_{bc}=4$, $\delta_{gs}=1$	4
12	96	`nt␣assessment␣`	1	`n`: $\delta_{bc}=1$, $\delta_{gs}=1$	1
13	97	`t␣assessment␣c`	10	ocurrencia; $gs[0]=10$	10
14	107	`nt␣concerns␣wi`	1	`␣`: $\delta_{bc}=10$, $\delta_{gs}=1$	10
15	117	`s␣within␣terti`	1	`r`: $\delta_{bc}=10$, $\delta_{gs}=1$	10
16	127	`ertiary␣educat`	1	`c`: $\delta_{bc}=10$, $\delta_{gs}=1$	10
17	137	`ucation.␣asses`	1	`s`: $\delta_{bc}=4$, $\delta_{gs}=1$	4
18	141	`ion.␣assessmen`	1	`m`: $\delta_{bc}=3$, $\delta_{gs}=1$	3
19	144	`.␣assessment␣c`	10	ocurrencia; $gs[0]=10$	10
20	154	`nt␣concerns␣ha`	1	`␣`: $\delta_{bc}=10$, $\delta_{gs}=1$	10
21	164	`s␣have␣largely`	1	`e`: $\delta_{bc}=2$, $\delta_{gs}=1$	2
22	166	`have␣largely␣r`	1	`y`: $\delta_{bc}=10$, $\delta_{gs}=1$	10
23	176	`ly␣revolved␣ar`	1	`␣`: $\delta_{bc}=10$, $\delta_{gs}=1$	10
24	186	`d␣around␣acade`	1	`a`: $\delta_{bc}=9$, $\delta_{gs}=1$	9
25	195	`academic␣integ`	3	`i`: $\delta_{bc}=8$, $\delta_{gs}=10$	10
26	205	`ntegrity␣issue`	1	`s`: $\delta_{bc}=4$, $\delta_{gs}=1$	4
27	209	`rity␣issues␣am`	1	`␣`: $\delta_{bc}=10$, $\delta_{gs}=1$	10
28	219	`s␣among␣studen`	1	`d`: $\delta_{bc}=10$, $\delta_{gs}=1$	10
29	229	`udents,␣such␣a`	1	`h`: $\delta_{bc}=10$, $\delta_{gs}=1$	10
30	239	`ch␣as␣plagiari`	1	`a`: $\delta_{bc}=9$, $\delta_{gs}=1$	9
31	248	`giarism␣and␣ch`	1	`␣`: $\delta_{bc}=10$, $\delta_{gs}=1$	10

En 26 de las 31 alineaciones decidió la regla del carácter malo y en 5 la del sufijo bueno. Como la

mayoría de los caracteres del texto no aparece en el patrón, en 22 de las 31 alineaciones el patrón saltó su longitud completa ($m = 10$). El resultado son 54 comparaciones, frente a las 300 de la fuerza bruta.

Complejidad. Preprocesamiento $O(m + \sigma)$, donde σ es el tamaño del alfabeto del patrón. En la búsqueda, el mejor caso es $\Theta(n/m)$ (cada intento salta m posiciones tras una sola comparación) y en texto natural el comportamiento es sublineal. El peor caso es $O(nm)$, por ejemplo $P = a^m$ sobre $T = a^n$: cada alineación compara m caracteres y avanza una posición. La regla de Galil [24] reduce ese caso a $O(n)$, pero no se exige.

string_matching/boyer_moore.py — tabla del sufijo bueno

```python
def good_suffix_table(pattern: str) -> tuple[list[int], list[int]]:
    m = len(pattern)
    shift = [0] * (m + 1)
    bpos = [0] * (m + 1)

    # Caso 1: el sufijo emparejado aparece en otra parte del patrón (sufijo bueno fuerte).
    i, j = m, m + 1
    bpos[i] = j
    while i > 0:
        while j <= m and pattern[i - 1] != pattern[j - 1]:
            if shift[j] == 0:
                shift[j] = j - i
            j = bpos[j]
        i -= 1
        j -= 1
        bpos[i] = j

    # Caso 2: sólo un prefijo del patrón coincide con una parte del sufijo emparejado.
    j = bpos[0]
    for i in range(m + 1):
        if shift[i] == 0:
            shift[i] = j
        if i == j:
            j = bpos[j]
    return shift, bpos
```

string_matching/boyer_moore.py — búsqueda (sin las líneas de traza)

```python
def boyer_moore_search(text: str, pattern: str, trace: bool = False, max_trace: int =
     DEFAULT_MAX_TRACE) -> MatchResult:
    validate(text, pattern)
    n, m = len(text), len(pattern)
    tracer = Tracer(trace, max_trace)

    t0 = time.perf_counter_ns()
    last = bad_character_table(pattern)
    gs, bpos = good_suffix_table(pattern)
    t1 = time.perf_counter_ns()

    positions: list[int] = []
    comparisons = 0
    alignments = 0
    bc_wins = gs_wins = 0
    s = 0
    while s <= n - m:
        alignments += 1
        j = m - 1
        while j >= 0:
            comparisons += 1
```

```
            match = pattern[j] == text[s + j]
            if not match:
                break
            j -= 1
        if j < 0:
            positions.append(s)
            shift = gs[0]
            gs_wins += 1
        else:
            c = text[s + j]
            bc = j - last.get(c, -1)
            gsv = gs[j + 1]
            if bc >= gsv:
                shift, rule = bc, "carácter malo"
                bc_wins += 1
            else:
                shift, rule = gsv, "sufijo bueno"
                gs_wins += 1
        s += shift
    t2 = time.perf_counter_ns()

    return MatchResult(
        algorithm="boyer_moore",
        pattern=pattern,
        text_length=n,
        positions=positions,
        comparisons=comparisons,
        preprocessing_ns=t1 - t0,
        search_ns=t2 - t1,
        alignments=alignments,
        extra={"bad_character_wins": bc_wins, "good_suffix_wins": gs_wins},
        tables={"bad_character": last, "good_suffix": gs, "border_positions": bpos},
        trace=tracer.events,
        trace_truncated=tracer.truncated,
    )
```

7.5. Algoritmo de Rabin-Karp

Idea. Rabin y Karp (1987) [25] comparan *huellas digitales* (hashes) en lugar de caracteres. Cada ventana de m caracteres se interpreta como un número en base d (el código Unicode de cada carácter es un «dígito») reducido módulo un primo q:

$$H(S) = \big(S[0]\,d^{m-1} + S[1]\,d^{m-2} + \cdots + S[m-1]\big) \bmod q. \tag{21}$$

El hash de la ventana siguiente se obtiene en $O(1)$ con el *rolling hash*, quitando el carácter que sale y agregando el que entra:

$$t_{s+1} = \big(d\,(t_s - T[s]\cdot h) + T[s+m]\big) \bmod q, \qquad h = d^{m-1} \bmod q. \tag{22}$$

Si $t_s = H(P)$, la ventana es candidata y se verifica carácter por carácter; si no coincide, se trata de una **colisión** (coincidencia espuria).

Cálculo numérico del caso de estudio. Se usa $d = 256$ y, para provocar colisiones a propósito, el primo pequeño $q = 13$. Primero, $h = 256^9 \bmod 13$: como $256 \equiv 9 \pmod{13}$ y $9^3 = 729 \equiv 1$, resulta $h = 9^9 \bmod 13 = 1$. El hash del patrón se calcula con la regla de Horner (tabla 24): $H \leftarrow (256\,H + \text{código}) \bmod 13$.

Tabla 24: Hash del patrón por la regla de Horner ($d = 256$, $q = 13$).

i	$P[i]$	**Código**	H **parcial**
0	a	97	6
1	s	115	0
2	s	115	11
3	e	101	5
4	s	115	4
5	s	115	8
6	m	109	12
7	e	101	1
8	n	110	2
9	t	116	4

$H(P) = 4$. Las primeras ventanas del texto y el cálculo iterativo de su sucesora son:

Tabla 25: Rolling hash en las primeras ventanas del texto.

s	**Ventana**	t_s	$t_s = H(P)$	**Cálculo de** t_{s+1}
0	the␣emerge	3	no	$(256\,(3 - 116 \cdot 1) + 110) \bmod 13 = 3$
1	he␣emergen	3	no	$(256\,(3 - 104 \cdot 1) + 99) \bmod 13 = 9$
2	e␣emergenc	9	no	$(256\,(9 - 101 \cdot 1) + 101) \bmod 13 = 1$
3	␣emergence	1	no	$(256\,(1 - 32 \cdot 1) + 32) \bmod 13 = 0$
4	emergence␣	0	no	$(256\,(0 - 101 \cdot 1) + 111) \bmod 13 = 8$
5	mergence␣o	8	no	$(256\,(8 - 109 \cdot 1) + 102) \bmod 13 = 12$

De las 258 ventanas, 19 tienen el mismo hash que el patrón (tabla 26). Sólo 2 son ocurrencias reales; las otras 17 son colisiones que la verificación descarta. Con un primo grande, $q = 10^9 + 7$ (el valor por defecto), hay 0 colisiones.

Tabla 26: Ventanas candidatas ($t_s = H(P) = 4$) con $q = 13$.

s	**Ventana**	t_s	**Verificación**
41	`telligence`	4	colisión (espuria)
53	`ai)␣such␣a`	4	colisión (espuria)
78	`parked␣sig`	4	colisión (espuria)
81	`ked␣signif`	4	colisión (espuria)
94	`nt␣assessm`	4	colisión (espuria)
97	`assessment`	4	**ocurrencia**
102	`sment␣conc`	4	colisión (espuria)
131	`y␣educatio`	4	colisión (espuria)
144	`assessment`	4	**ocurrencia**
149	`sment␣conc`	4	colisión (espuria)
170	`argely␣rev`	4	colisión (espuria)
178	`evolved␣ar`	4	colisión (espuria)
185	`␣around␣ac`	4	colisión (espuria)
198	`mic␣integr`	4	colisión (espuria)
202	`integrity␣`	4	colisión (espuria)
242	`␣plagiaris`	4	colisión (espuria)
246	`giarism␣an`	4	colisión (espuria)
251	`sm␣and␣che`	4	colisión (espuria)
256	`d␣cheating`	4	colisión (espuria)

Complejidad. Preprocesamiento $\Theta(m)$ (h y $H(P)$). Si los hashes se distribuyen de manera uniforme, la probabilidad de colisión de una ventana es $\approx 1/q$ y el tiempo esperado de búsqueda es

$$E[T(n,m)] = O\left(n + m + \frac{(n-m+1)\,m}{q} + \text{occ} \cdot m\right), \tag{23}$$

es decir, $O(n+m)$ con q grande. El peor caso es $O(nm)$, cuando casi todas las ventanas coinciden en hash (muchas ocurrencias, como $P = a^m$ en $T = a^n$, o un q muy pequeño). La memoria es $O(1)$.

`string_matching/rabin_karp.py — búsqueda (sin las líneas de traza)`

```python
def rabin_karp_search(
    text: str,
    pattern: str,
    base: int = DEFAULT_BASE,
    modulus: int = DEFAULT_MODULUS,
    trace: bool = False,
    max_trace: int = DEFAULT_MAX_TRACE,
) -> MatchResult:
    validate(text, pattern)
    if base < 2 or modulus < 2:
        raise ValueError("La base y el módulo deben ser >= 2")
    n, m = len(text), len(pattern)
    tracer = Tracer(trace, max_trace)

    t0 = time.perf_counter_ns()
    h = 1
    for _ in range(m - 1):  # h = d^(m-1) mod q por multiplicaciones sucesivas
        h = (h * base) % modulus
    p_hash = 0
    pattern_steps = []
    for idx in range(m):
        p_hash = (p_hash * base + ord(pattern[idx])) % modulus
    t1 = time.perf_counter_ns()
```

```
    positions: list[int] = []
    comparisons = 0
    hash_hits = spurious = 0
    alignments = 0
    if n >= m:
        t_hash = 0
        for idx in range(m):
            t_hash = (t_hash * base + ord(text[idx])) % modulus
        for s in range(n - m + 1):
            alignments += 1
            equal = t_hash == p_hash
            verified = None
            if equal:
                hash_hits += 1
                j = 0
                while j < m:
                    comparisons += 1
                    if text[s + j] != pattern[j]:
                        break
                    j += 1
                verified = j == m
                if verified:
                    positions.append(s)
                else:
                    spurious += 1
            next_hash = None
            if s < n - m:
                out_c, in_c = text[s], text[s + m]
                next_hash = (base * (t_hash - ord(out_c) * h) + ord(in_c)) % modulus
            if verified:
            elif equal:
            if next_hash is not None:
                t_hash = next_hash
    t2 = time.perf_counter_ns()

    return MatchResult(
        algorithm="rabin_karp",
        pattern=pattern,
        text_length=n,
        positions=positions,
        comparisons=comparisons,
        preprocessing_ns=t1 - t0,
        search_ns=t2 - t1,
        alignments=alignments,
        extra={"hash_hits": hash_hits, "spurious_hits": spurious, "base": base, "modulus": modulus},
        tables={"h": h, "pattern_hash": p_hash, "pattern_hash_steps": pattern_steps},
        trace=tracer.events,
        trace_truncated=tracer.truncated,
    )
```

7.6. Análisis de complejidad y contraste empírico

Tabla 27: Complejidad asintótica teórica.

Algoritmo	Preprocesamiento	Mejor caso	Caso típico	Peor caso	Memoria
Fuerza bruta	—	$\Theta(n)$	$\Theta(n)^*$	$O(nm)$	$O(1)$
KMP	$\Theta(m)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$O(m)$
Boyer-Moore	$O(m+\sigma)$	$\Theta(n/m)$	sublineal	$O(nm)$	$O(m+\sigma)$
Rabin-Karp	$\Theta(m)$	$\Theta(n)$	$O(n+m)$	$O(nm)$	$O(1)$

* En texto natural casi todas las alineaciones fallan en el primer o segundo carácter.

Metodología. El benchmark (`string_matching/benchmark.py`) mide la mediana de 5 repeticiones de los tiempos de preprocesamiento y de búsqueda (Python 3.11.14, arm64) y el número de comparaciones de caracteres, una métrica independiente de la máquina. Se hicieron tres experimentos: (A) variar m con $n = 113\,052$ caracteres, usando patrones extraídos del propio corpus; (B) variar n replicando el corpus de 1 a 16 veces (28 263 caracteres por copia); (C) construir los peores casos teóricos sobre $T = a^n$ con $n = 20\,000$.

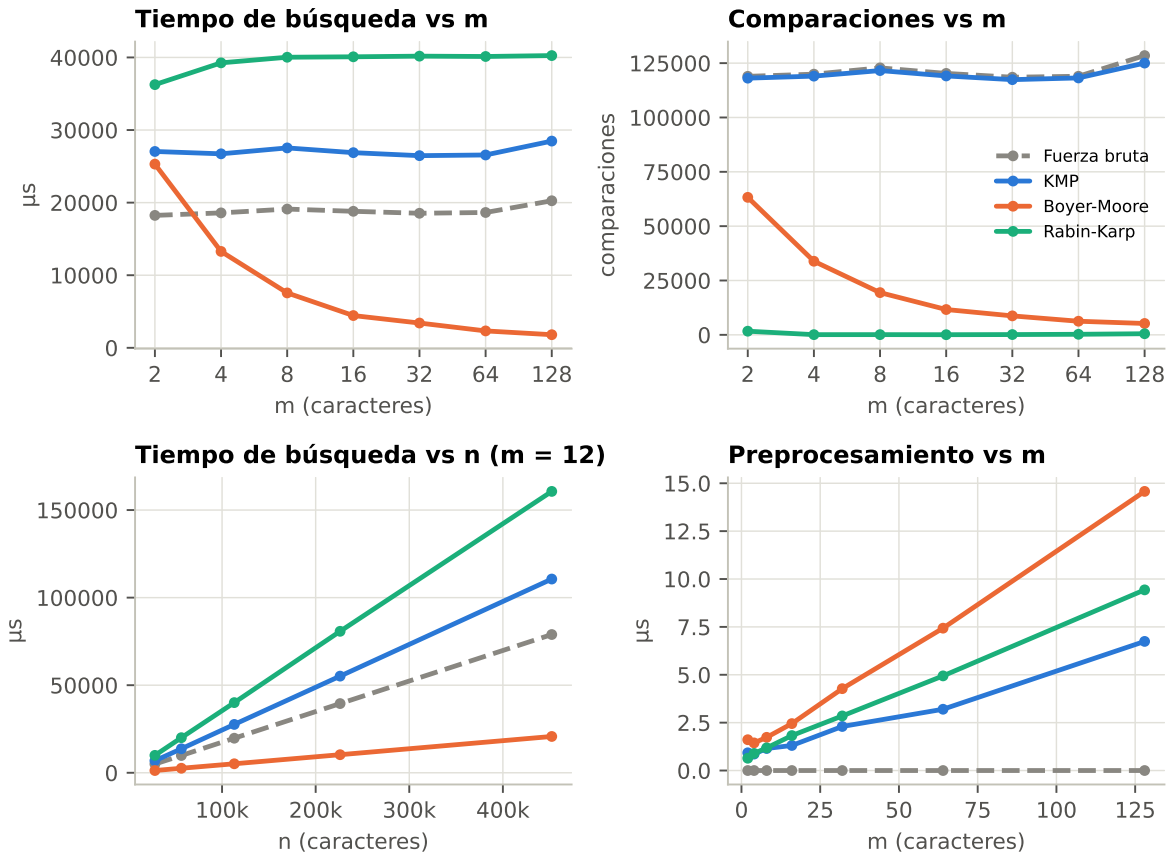


Figura 15: Resultados empíricos: tiempo de búsqueda y comparaciones según m, tiempo según n y tiempo de preprocesamiento según m.

Tabla 28: Ajuste lineal del tiempo de búsqueda frente a n (experimento B): costo por carácter y coeficiente R^2.

Algoritmo	ns por carácter	R^2
Fuerza bruta	174.5	1.0000
KMP	244.8	1.0000
Boyer-Moore	45.9	1.0000
Rabin-Karp	355.7	1.0000

Tabla 29: Comparaciones en los peores casos teóricos ($T = a^n$, $n = 20\,000$).

Patrón	m	Fuerza bruta	KMP	Boyer-Moore	Rabin-Karp
$a^{m-1}b$	5	99 980	39 996	19 996	0
a^m	5	99 980	20 000	99 980	99 980
$a^{m-1}b$	10	199 910	39 991	19 991	0
a^m	10	199 910	20 000	199 910	199 910
$a^{m-1}b$	20	399 620	39 981	19 981	0
a^m	20	399 620	20 000	399 620	399 620
$a^{m-1}b$	40	798 440	39 961	19 961	0
a^m	40	798 440	20 000	798 440	798 440
$a^{m-1}b$	80	1 593 680	39 921	19 921	0
a^m	80	1 593 680	20 000	1 593 680	1 593 680

Tabla 30: Colisiones de Rabin-Karp según el módulo q (patrón `learning` sobre los 20 abstracts, $n \approx 28\,000$).

q	Hashes iguales	Colisiones	Ocurrencias	Comparaciones
13	2206	2166	40	2551
101	320	280	40	607
997	64	24	40	344
10 007	41	1	40	321
1 000 000 007	40	0	40	320

Contraste entre teoría y observación.

1. **Linealidad en n.** Los cuatro algoritmos crecen linealmente con el tamaño del texto ($R^2 \approx 1$ en la tabla 28), como predice la teoría en texto natural.
2. **Boyer-Moore es sublineal y mejora con m.** Sus comparaciones bajan de 63 258 con $m = 2$ a 5 229 con $m = 128$, y su tiempo de búsqueda se reduce 14.0 veces: los saltos crecen con la longitud del patrón, en línea con el mejor caso $\Theta(n/m)$. Es el más rápido en todos los experimentos con texto natural.
3. **KMP realiza $\approx n$ comparaciones, independientemente de m** (118 094 con $m = 2$). En el peor caso $P = a^{m-1}b$ hace a lo sumo $2n$ (39 996 con $n = 20\,000$, tabla 29), mientras la fuerza bruta crece como $(n - m + 1)\,m$ (1.59 millones con $m = 80$). Es el único de los tres con garantía lineal en el peor caso: con $P = a^m$, Boyer-Moore y Rabin-Karp degeneran a $O(nm)$, igual que la fuerza bruta.
4. **Las constantes importan.** En Python puro, la fuerza bruta resulta más rápida que KMP en texto natural (174.5 frente a 244.8 ns por carácter): casi todas las alineaciones fallan en el primer ca-

rácter, y KMP paga por cada carácter la lógica de la función de prefijos. Rabin-Karp es el más lento en tiempo (355.7 ns por carácter) pese a hacer casi cero comparaciones, porque cada ventana requiere varias operaciones aritméticas modulares. La notación O describe el crecimiento, no el costo constante por operación.

5. **Colisiones de Rabin-Karp.** El número de colisiones observado coincide con la estimación teórica $(n - m + 1)/q$: con $q = 13$ se esperan $\approx 2\,174$ y se observaron 2 166; con $q = 101$, ≈ 280 frente a 280. Con $q = 10^9 + 7$ no hubo ninguna (tabla 30). Las colisiones no afectan la corrección, gracias a la verificación, pero sí el tiempo.

6. **Preprocesamiento frente a búsqueda.** El preprocesamiento es $\Theta(m)$ en los tres algoritmos y no supera unos pocos microsegundos, frente a decenas de milisegundos de búsqueda; su impacto sólo sería relevante con textos muy cortos o muchos patrones distintos. Boyer-Moore tiene el preprocesamiento más costoso (dos tablas), pero lo amortiza con creces en la búsqueda.

Tabla 31: Búsqueda de `"assessment"` en los 20 abstracts (28 244 caracteres; 37 ocurrencias en 11 artículos).

Algoritmo	Ocurr.	Comparaciones	Alineaciones	Preproc. (µs)	Búsqueda (µs)
KMP	37	30 248	27 824	16.2	7067.7
Boyer-Moore	37	4 232	3 495	42.3	1620.0
Rabin-Karp	37	370	28 064	24.6	10100.8

Conclusión del Requerimiento 3

Los tres algoritmos encuentran exactamente las mismas ocurrencias (la aplicación lo verifica en cada búsqueda). Para buscar palabras o frases en abstracts, **Boyer-Moore** es la mejor opción: compara una fracción del texto y se acelera con patrones largos. **KMP** ofrece la única garantía lineal en el peor caso y conviene con alfabetos pequeños o patrones repetitivos. **Rabin-Karp** hace muy pocas comparaciones con un q grande y se extiende de forma natural a la búsqueda simultánea de varios patrones, pero en Python su aritmética modular lo hace el más lento.

8. Requerimiento 4: Despliegue y documentación técnica

La aplicación puede usarse de tres formas: por consola, como servicio web con interfaz gráfica y como contenedor Docker listo para publicarse en la nube.

8.1. Ejecución local por consola

El archivo `main.py` ofrece un menú interactivo, al estilo de los proyectos de referencia del curso:

Tabla 32: Opciones del menú por consola.

Opción	Requerimiento	Acción
1	Corpus	Extraer/actualizar el corpus desde los PDF (`data/processed/corpus.json`).
2	Corpus	Listar los artículos con año y autores.
3	R1	Comparar artículos (IDs separados por coma); Levenshtein por palabras o caracteres.
4	R2	Agrupamiento con TF-IDF o SBERT; imprime CPCC y siluetas y guarda los dendrogramas PNG y las métricas CSV en `outputs/clustering/`.
5	R3	Buscar un patrón en abstracts, títulos, autores y palabras clave; muestra el contexto y las métricas de cada algoritmo.
6	R3	Ejecutar el benchmark completo.
7	R4	Iniciar la aplicación web (API + interfaz) en `http://localhost:8000`.

Instalación y ejecución:

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python main.py
```

8.2. API REST

`bibliometria/api/main.py` expone los requerimientos como servicios JSON. FastAPI valida automáticamente las entradas (por ejemplo, al menos dos artículos para comparar) y publica la documentación interactiva OpenAPI en `/docs` (figura 22).

Tabla 33: Endpoints de la API.

Método	Ruta	Descripción
`GET`	`/api/health, /api/meta`	Estado del servicio y catálogo de algoritmos.
`GET`	`/api/corpus[/{id}]`	Artículos con título, autores, abstract y metadatos.
`GET`	`/api/corpus/{id}/preprocessing`	Etapas del preprocesamiento de un abstract.
`POST`	`/api/similarity/compare`	R1: similitud de todos los pares de una selección (6 algoritmos).
`GET`	`/api/similarity/explain`	R1: demostración paso a paso de un par (matrices, pesos, vectores).
`GET`	`/api/similarity/matrix`	R1: matriz 20×20 de un algoritmo.
`GET`	`/api/clustering`	R2: fusiones, dendrogramas, CPCC, siluetas y grupos (TF-IDF o SBERT).
`POST`	`/api/search`	R3: búsqueda en el corpus con los tres algoritmos y sus métricas.
`POST`	`/api/search/trace`	R3: ejecución instrumentada paso a paso sobre un texto.
`GET`	`/api/search/benchmark`	R3: resultados del benchmark empírico.
`GET`	`/informe.pdf`	Este documento, descargable desde la interfaz.

8.3. Interfaz web

La interfaz (`frontend/`) es una aplicación de una sola página construida con React, TypeScript y Vite, con componentes de `shadcn/ui` y Tailwind CSS. Las gráficas se dibujan con Recharts, las fórmulas con KaTeX, y los dendrogramas, mapas de calor y matrices de programación dinámica con SVG y tablas propias. La paleta de colores se validó para daltonismo; la interfaz tiene modo claro y oscuro, se adapta

a teléfonos y respeta la preferencia de movimiento reducido. Cada gráfica tiene su tabla de valores equivalente. Hay una página por requerimiento:

- **Corpus**: estadísticas, gráficas y tabla filtrable de los 20 artículos con su ficha (abstract, autores, palabras clave, DOI).
- **Similitud textual**: selector múltiple de artículos, resultados por par, mapas de calor, demostración paso a paso, matriz del corpus completo y fichas de los algoritmos.
- **Agrupamiento**: representación (TF-IDF o SBERT), criterio de enlace y número de grupos k interactivos; dendrograma con corte, métricas, curva de silueta, composición de los grupos y tabla de fusiones.
- **Búsqueda de patrones**: búsqueda con resaltado del contexto y cruce con título y autores, métricas por algoritmo, reproductor paso a paso con las tablas auxiliares (π, carácter malo, sufijo bueno, hashes) y resultados del benchmark.
- **Arquitectura y despliegue**: diagrama de capas, instrucciones de ejecución y despliegue, equipo y descarga de este informe.

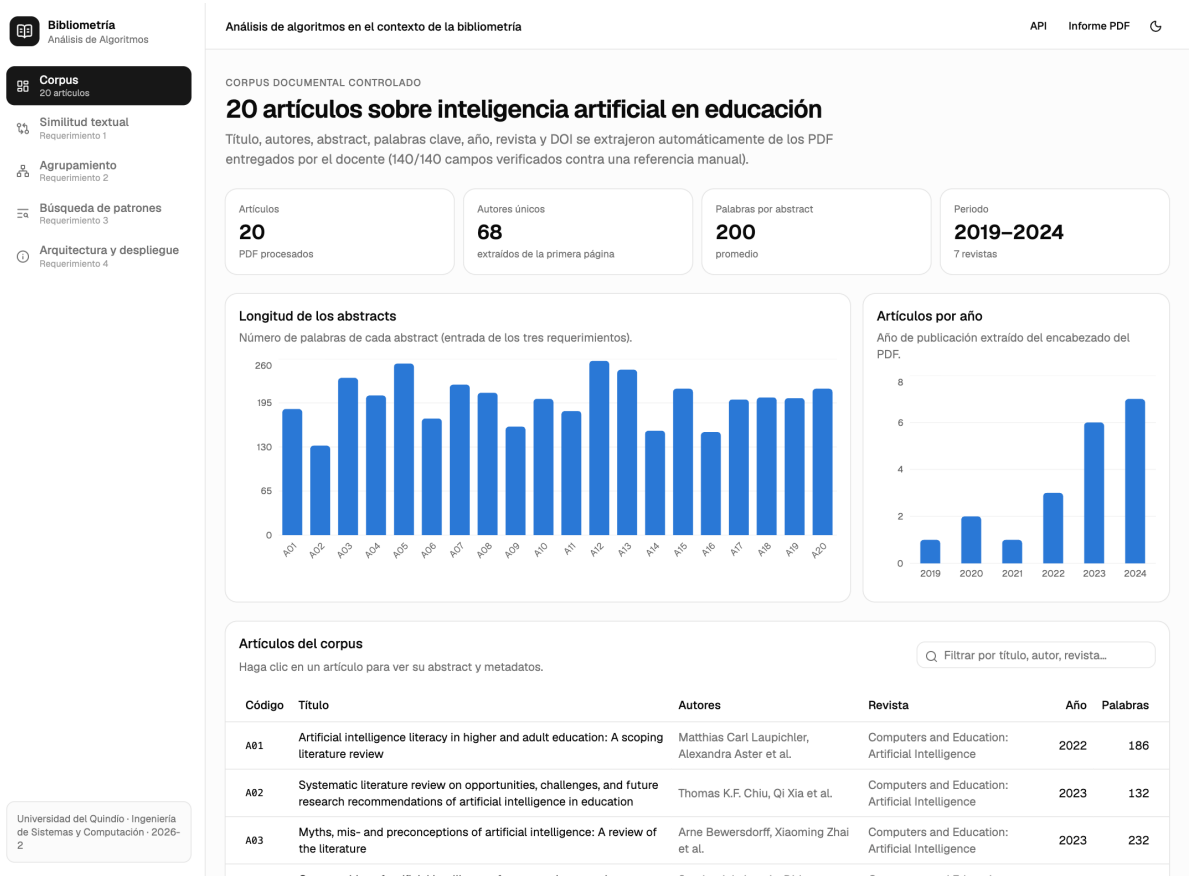


Figura 16: Interfaz web: página del corpus.

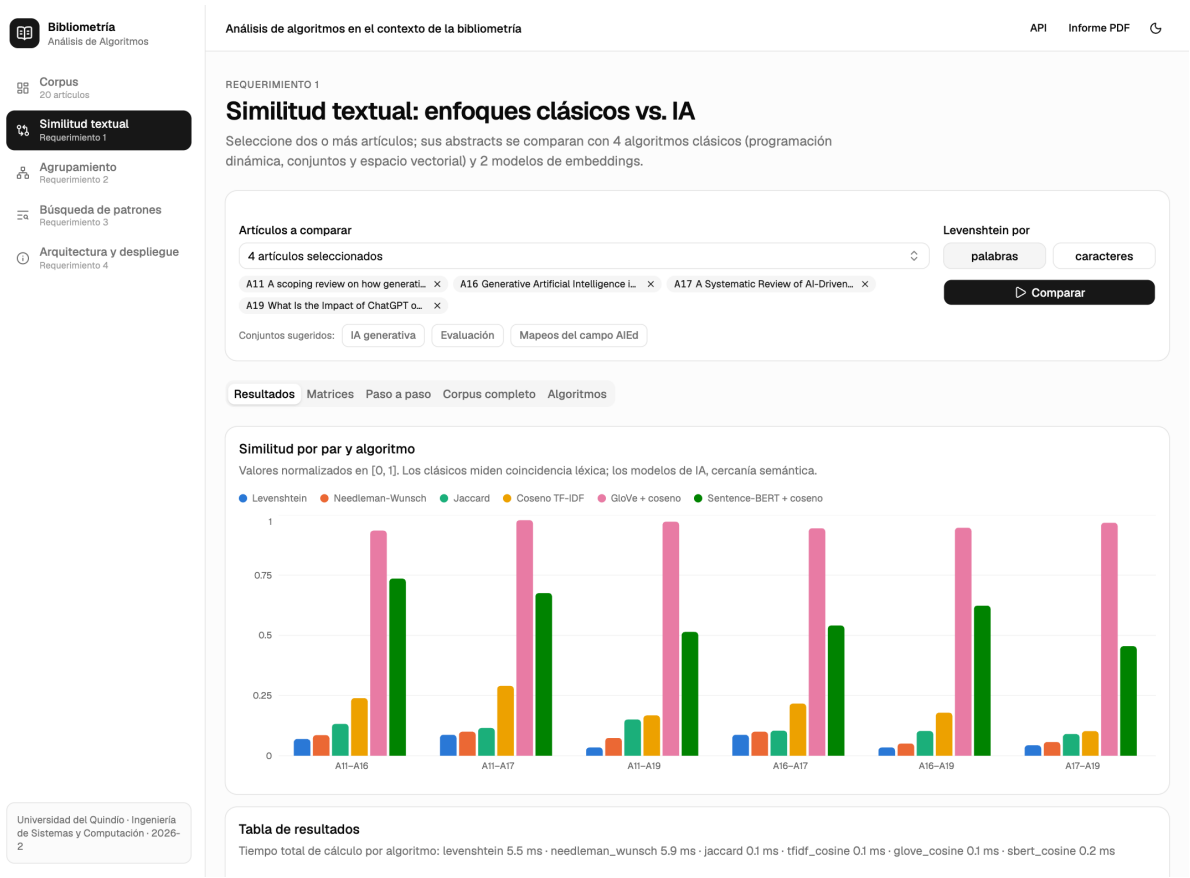


Figura 17: Interfaz web: comparación de cuatro artículos con los seis algoritmos (R1).

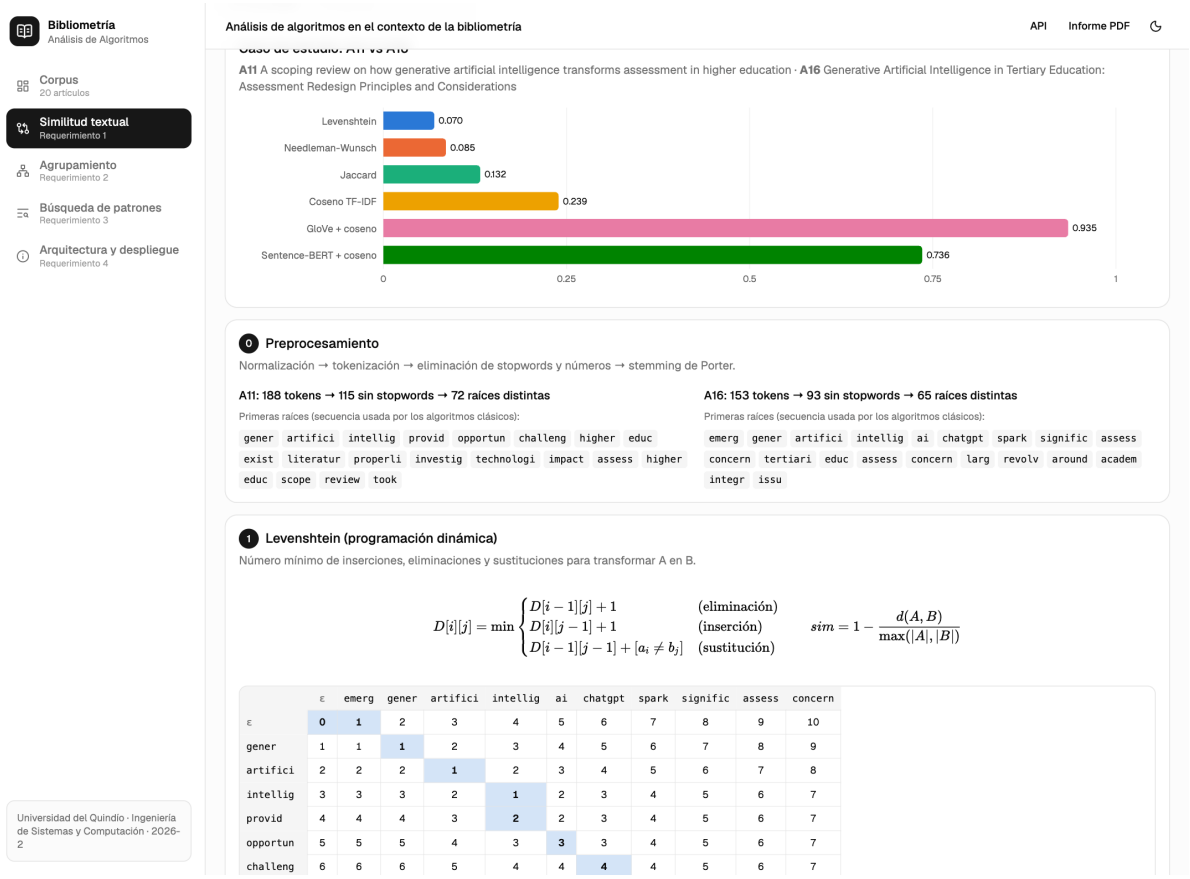


Figura 18: Interfaz web: demostración paso a paso del caso de estudio (R1).

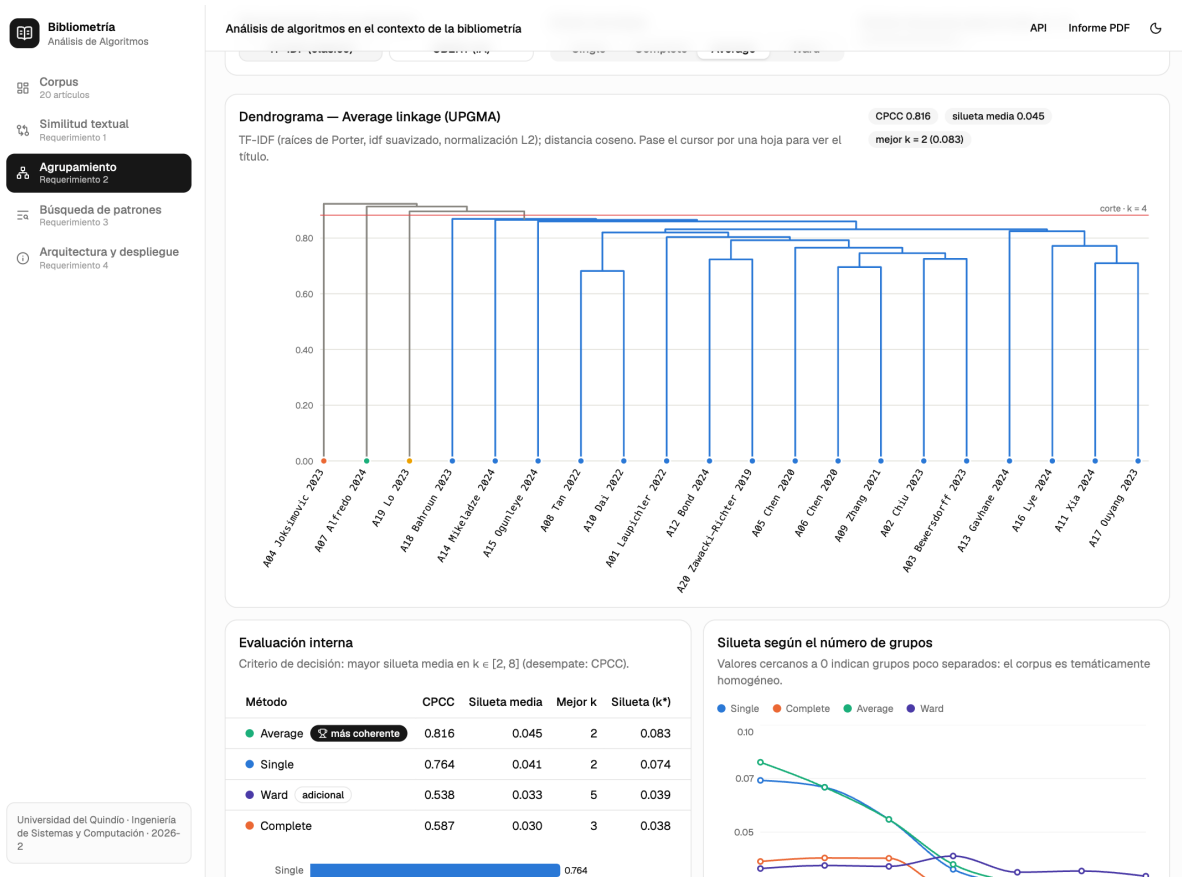


Figura 19: Interfaz web: dendrograma interactivo y evaluación interna (R2).

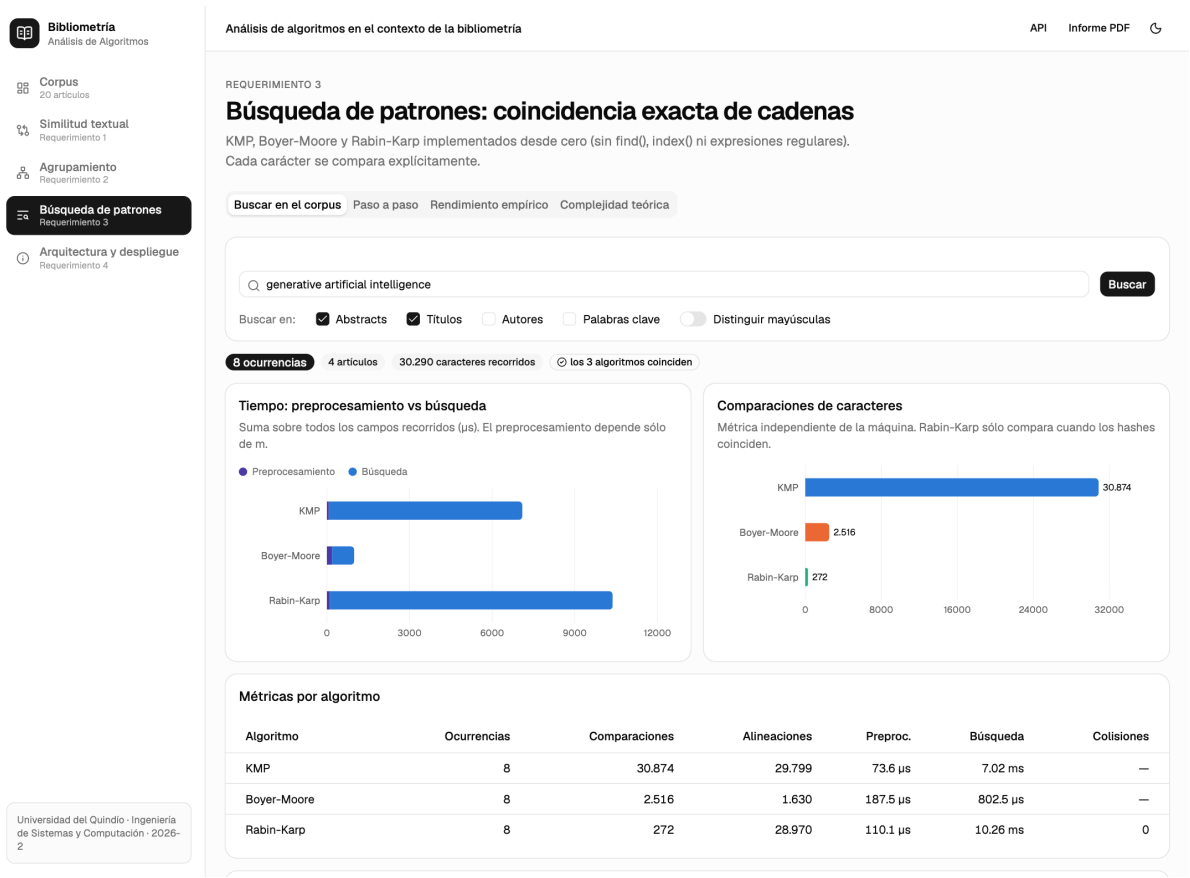


Figura 20: Interfaz web: búsqueda de «generative artificial intelligence» en abstracts y títulos (R3).

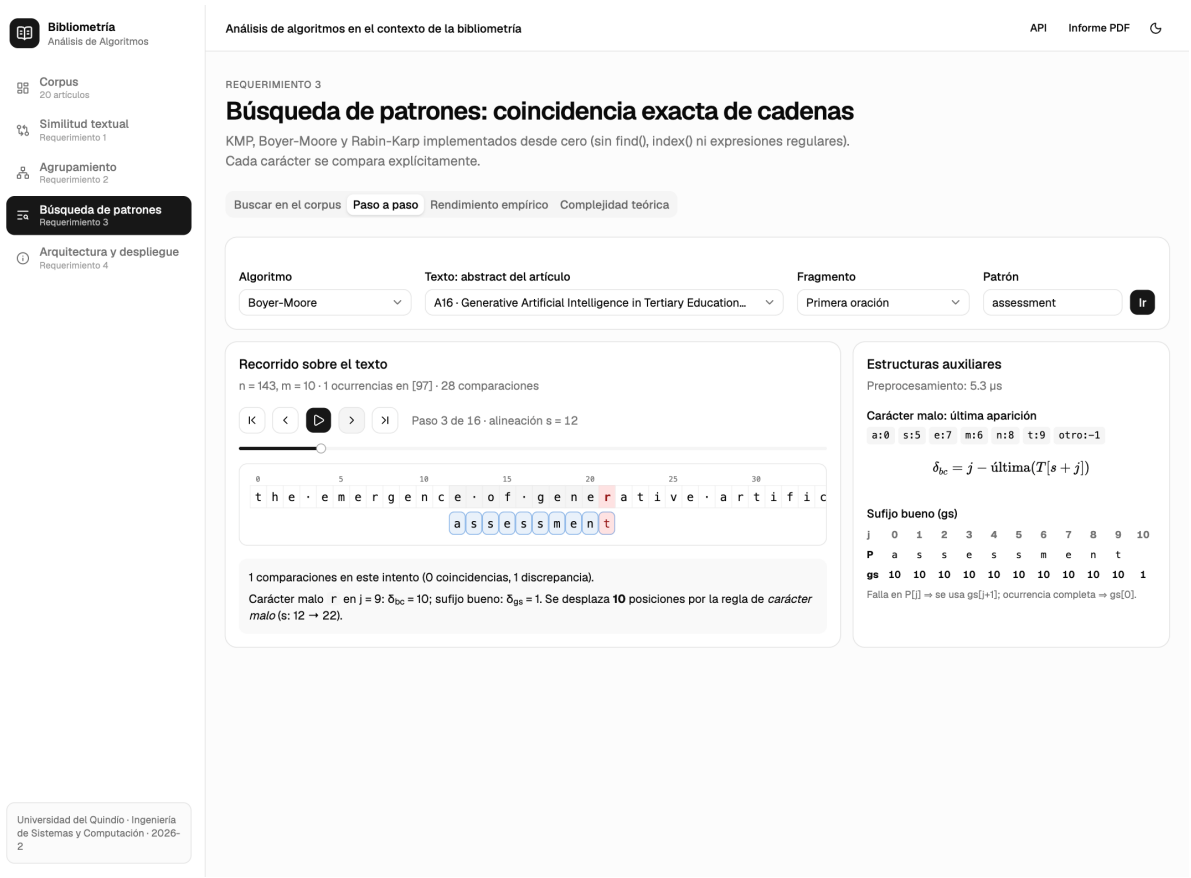


Figura 21: Interfaz web: reproductor paso a paso de Boyer-Moore con sus tablas heurísticas (R3).



Figura 22: Documentación interactiva de la API (OpenAPI) en `/docs`.

8.4. Despliegue con Docker en OVHcloud

El `Dockerfile` tiene dos etapas: la primera compila la interfaz con Node; la segunda instala sólo las dependencias de ejecución de Python (FastAPI, Uvicorn, PyMuPDF y Matplotlib; sin PyTorch, gracias a los embeddings precalculados), copia el paquete, los datos, la interfaz compilada y este informe en PDF, y ejecuta la aplicación con un usuario sin privilegios y una verificación de salud sobre `/api/health`. La imagen resultante (unos 700 MB; PyTorch y los modelos le agregarían más de 1 GB) sirve todo el sistema por un único puerto, y se construyó y probó localmente.

La aplicación se despliega en un servidor de **OVHcloud** con el dominio propio del equipo. El archivo `docker-compose.yml` define dos servicios: `app`, la imagen anterior, accesible sólo desde el propio servidor; y `caddy`, un proxy inverso que publica la aplicación en los puertos 80 y 443 y obtiene y renueva automáticamente el certificado HTTPS de Let's Encrypt para el dominio configurado en el archivo `.env`:

```
# En el servidor, con el registro DNS del dominio apuntando a su IP pública
cp .env.example .env                              # DOMAIN=<dominio del equipo>
docker compose --profile produccion up -d --build   # app + Caddy con HTTPS
```

Para ejecutarla localmente basta con `docker compose up -d --build`, que publica la aplicación en `http://localhost:8000`.

8.5. Pruebas y validación

La corrección de las implementaciones se verificó con cinco tipos de pruebas automatizadas:

Tabla 34: Estrategia de pruebas (177 passed in 10.36s con `pytest`; 10 pruebas de extremo a extremo con Playwright).

Tipo	Qué se verifica
Unitarias con valores a mano	Distancias conocidas (`kitten`/`sitting` = 3), tablas π de Cormen et al., tabla del sufijo bueno, hashes de Rabin-Karp calculados a mano, fórmulas de Lance-Williams, raíces de Porter.
De oráculo	Las implementaciones propias reproducen los resultados de librerías de referencia: Levenshtein = RapidFuzz; TF-IDF y coseno = scikit-learn (10^{-9}); fusiones, alturas, CPCC, cortes y geometría del dendrograma = SciPy; silueta = scikit-learn (10^{-12}); stemmer = NLTK.
De propiedades (Hypothesis)	Miles de entradas aleatorias: KMP, Boyer-Moore y Rabin-Karp (incluso con $q = 7$ o 13) coinciden con la fuerza bruta; Levenshtein cumple simetría, identidad, desigualdad triangular y cotas; Needleman-Wunsch es óptimo frente a un oráculo recursivo; KMP hace a lo sumo $2n$ comparaciones.
De extracción	Los 7 campos de los 20 PDF coinciden con la referencia manual (140/140).
De integración y de flujo	Todos los endpoints con entradas válidas e inválidas; reconstrucción del corpus desde los PDF idéntica a la publicada; recorrido de un usuario por los tres requerimientos.
De extremo a extremo	Playwright abre la aplicación en Chrome (escritorio y móvil), selecciona artículos, compara, cambia criterios de agrupamiento, busca patrones, reproduce trazas y verifica que no haya errores en la consola.

```
pip install -r requirements-dev.txt && pytest          # 177 pruebas
cd frontend && npx playwright test                        # 10 pruebas E2E
```

8.6. Documentación técnica

La documentación del proyecto se compone de: (1) este informe, con el diseño, la arquitectura y la explicación técnica de cada requerimiento; (2) el `README.md` con la instalación, el uso, las pruebas y el despliegue; (3) la documentación OpenAPI de la API, y (4) los *docstrings* de cada módulo, que incluyen la definición matemática y la complejidad de cada algoritmo.

9. Declaración de uso de inteligencia artificial generativa

En el proyecto intervienen dos usos distintos de la IA, que conviene separar.

9.1. IA como componente de la solución

El Requerimiento 1 exige dos enfoques de similitud basados en IA. Se usan dos modelos preentrenados públicos: GloVe (`glove-wiki-gigaword-100`) y Sentence-BERT (`all-MiniLM-L6-v2`). Su único papel es *generar los vectores* de los abstracts. La similitud del coseno, la distancia euclidiana, el promedio de vectores y todo el análisis posterior se implementaron con código propio. Se eligieron por tres razones: (i) representan dos generaciones de modelos de lenguaje (embeddings estáticos frente a contextuales), lo que enriquece la comparación con los métodos clásicos; (ii) son modelos abiertos, documentados y reproducibles, y (iii) son livianos y pueden ejecutarse en un computador personal sin servicios de pago.

9.2. IA generativa como herramienta de apoyo al desarrollo

Durante el desarrollo se utilizó **Claude** (Anthropic), mediante la herramienta **Claude Code**, como asistente. Su uso comprendió:

- consulta de información sobre los algoritmos, las librerías y las herramientas del proyecto;
- apoyo en la implementación y la organización del código de los módulos, de la API, de la interfaz web y de las pruebas automatizadas;
- apoyo en la redacción y la diagramación de este documento;
- revisión de la consistencia entre el código, los resultados y el texto.

La corrección de los resultados no depende de la herramienta de IA y se verificó con los siguientes mecanismos:

- **Implementación explícita y verificable.** Ningún algoritmo solicitado se delega en librerías. Todos están escritos con estructuras básicas, y su comportamiento puede seguirse línea a línea en el código y en las trazas paso a paso de la aplicación.
- **Verificación independiente.** La corrección no descansa en la herramienta de IA: 177 pruebas automatizadas comparan cada implementación con librerías de referencia (SciPy, scikit-learn, NLTK, RapidFuzz), con valores calculados a mano y con miles de casos aleatorios.
- **Reproducibilidad.** Los resultados de este documento se obtienen ejecutando la aplicación y pueden regenerarse con los comandos del anexo B.
- **Responsabilidad del equipo.** Los integrantes revisaron el código, los resultados y el análisis, y asumen la responsabilidad del contenido entregado y de su sustentación.

Esta declaración se ajustará a los lineamientos sobre el uso de IA que entregue el docente.

10. Conclusiones

1. **La extracción automática es viable con reglas geométricas.** Con la posición, el tamaño de fuente y las banderas de cada línea, y anclando el título al nombre del archivo, se extrajeron correctamente los 140 campos de los 20 PDF, a pesar de las cinco diagramaciones editoriales distintas.
2. **Similitud: léxica frente a semántica.** Los algoritmos de edición (Levenshtein, Needleman-Wunsch) miden alineamientos secuenciales y dan valores casi nulos entre abstracts distintos; sirven para detectar copias, no afinidad temática. Jaccard y, sobre todo, TF-IDF son líneas base léxicas eficientes, interpretables y lineales. Sentence-BERT es el método más fiable para similitud semántica: ubica el caso de estudio (A11–A16, ambos sobre IA generativa y evaluación) entre los tres pares más similares del corpus, mientras que los métodos léxicos lo ubican entre los puestos 21 (TF-IDF) y 66 (Jaccard) de 190. GloVe con promedio sufre anisotropía y desconoce términos recientes como *chatgpt*: no todo modelo de IA es mejor que uno clásico bien ponderado.
3. **Agrupamiento: average linkage es el criterio más coherente** de los tres exigidos según la silueta media y la correlación cofenética, con TF-IDF y con SBERT. Las siluetas bajas reflejan un corpus temáticamente homogéneo, y la elección de la representación pesa más que la del criterio: con SBERT los grupos coinciden con subtemas reconocibles (IA generativa, mapeos del campo, alfabetización y competencias, aprendizaje apoyado por IA).
4. **Búsqueda de patrones: la teoría se confirma, con matices.** Boyer-Moore es sublineal en texto natural y se acelera con patrones largos; KMP es el único con garantía lineal en el peor caso ($\leq 2n$

comparaciones); Rabin-Karp casi no compara caracteres con un primo grande, y sus colisiones con primos pequeños coinciden con la predicción $(n-m+1)/q$. A la vez, en Python las constantes ocultas por la notación O deciden el tiempo real: la fuerza bruta supera a KMP en texto natural y Rabin-Karp es el más lento por su aritmética modular.

5. **Transparencia y verificación.** Implementar cada algoritmo desde cero, instrumentarlo con trazas y contrastarlo con librerías de referencia permitió demostrar su funcionamiento interno con casos concretos y, a la vez, tener evidencia objetiva de su corrección: 177 pruebas automatizadas y 10 pruebas de extremo a extremo.

Referencias

[1] V. I. Levenshtein, «Binary codes capable of correcting deletions, insertions, and reversals», *Soviet Physics Doklady*, vol. 10, n.º 8, pp. 707–710, 1966.

[2] R. A. Wagner y M. J. Fischer, «The string-to-string correction problem», *Journal of the ACM*, vol. 21, n.º 1, pp. 168–173, 1974.

[3] S. B. Needleman y C. D. Wunsch, «A general method applicable to the search for similarities in the amino acid sequence of two proteins», *Journal of Molecular Biology*, vol. 48, n.º 3, pp. 443–453, 1970.

[4] P. Jaccard, «The distribution of the flora in the alpine zone», *New Phytologist*, vol. 11, n.º 2, pp. 37–50, 1912.

[5] G. Salton y C. Buckley, «Term-weighting approaches in automatic text retrieval», *Information Processing & Management*, vol. 24, n.º 5, pp. 513–523, 1988.

[6] C. D. Manning, P. Raghavan y H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 2008.

[7] M. F. Porter, «An algorithm for suffix stripping», *Program*, vol. 14, n.º 3, pp. 130–137, 1980.

[8] T. Mikolov, K. Chen, G. Corrado y J. Dean, «Efficient estimation of word representations in vector space», arXiv:1301.3781, 2013.

[9] J. Pennington, R. Socher y C. D. Manning, «GloVe: Global vectors for word representation», en *Proc. EMNLP*, 2014, pp. 1532–1543.

[10] A. Vaswani *et al.*, «Attention is all you need», en *Advances in Neural Information Processing Systems 30*, 2017.

[11] N. Reimers e I. Gurevych, «Sentence-BERT: Sentence embeddings using Siamese BERT-networks», en *Proc. EMNLP-IJCNLP*, 2019, pp. 3982–3992.

[12] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang y M. Zhou, «MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers», en *Advances in Neural Information Processing Systems 33*, 2020.

[13] S. Arora, Y. Liang y T. Ma, «A simple but tough-to-beat baseline for sentence embeddings», en *Proc. ICLR*, 2017.

[14] J. Mu y P. Viswanath, «All-but-the-top: Simple and effective postprocessing for word representations», en *Proc. ICLR*, 2018.

[15] S. C. Johnson, «Hierarchical clustering schemes», *Psychometrika*, vol. 32, n.º 3, pp. 241–254, 1967.

[16] G. N. Lance y W. T. Williams, «A general theory of classificatory sorting strategies: 1. Hierarchical systems», *The Computer Journal*, vol. 9, n.º 4, pp. 373–380, 1967.

[17] J. H. Ward, «Hierarchical grouping to optimize an objective function», *Journal of the American Statistical Association*, vol. 58, n.º 301, pp. 236–244, 1963.

[18] R. R. Sokal y F. J. Rohlf, «The comparison of dendrograms by objective methods», *Taxon*, vol. 11, n.º 2, pp. 33–40, 1962.

[19] P. J. Rousseeuw, «Silhouettes: A graphical aid to the interpretation and validation of cluster analysis», *Journal of Computational and Applied Mathematics*, vol. 20, pp. 53–65, 1987.

[20] F. Murtagh y P. Contreras, «Algorithms for hierarchical clustering: An overview», *WIREs Data Mining and Knowledge Discovery*, vol. 2, n.º 1, pp. 86–97, 2012.

[21] D. Müllner, «Modern hierarchical, agglomerative clustering algorithms», arXiv:1109.2378, 2011.

[22] D. E. Knuth, J. H. Morris y V. R. Pratt, «Fast pattern matching in strings», *SIAM Journal on Computing*, vol. 6, n.º 2, pp. 323–350, 1977.

[23] R. S. Boyer y J. S. Moore, «A fast string searching algorithm», *Communications of the ACM*, vol. 20, n.º 10, pp. 762–772, 1977.

[24] Z. Galil, «On improving the worst case running time of the Boyer-Moore string matching algorithm», *Communications of the ACM*, vol. 22, n.º 9, pp. 505–508, 1979.

[25] R. M. Karp y M. O. Rabin, «Efficient randomized pattern-matching algorithms», *IBM Journal of Research and Development*, vol. 31, n.º 2, pp. 249–260, 1987.

[26] T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein, *Introduction to Algorithms*, 4.ª ed. MIT Press, 2022.

[27] D. Gusfield, *Algorithms on Strings, Trees, and Sequences*. Cambridge University Press, 1997.

[28] S. Ramírez, «FastAPI». [En línea]. Disponible: https://fastapi.tiangolo.com

[29] Artifex Software, «PyMuPDF documentation». [En línea]. Disponible: https://pymupdf.readthedocs.io

[30] UKPLab, «Sentence Transformers documentation». [En línea]. Disponible: https://www.sbert.net

[31] shadcn, «shadcn/ui». [En línea]. Disponible: https://ui.shadcn.com

[32] Recharts Group, «Recharts». [En línea]. Disponible: https://recharts.org

[33] P. Virtanen *et al.*, «SciPy 1.0: Fundamental algorithms for scientific computing in Python», *Nature Methods*, vol. 17, pp. 261–272, 2020.

[34] F. Pedregosa *et al.*, «Scikit-learn: Machine learning in Python», *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

A. Corpus completo

Tabla 35: Los 20 artículos del corpus con los metadatos extraídos automáticamente.

Cód.	Título	Autores	Año	Revista	Pal.
A01	Artificial intelligence literacy in higher and adult education: A scoping literature review	Matthias Carl Laupichler et al.	2022	Computers and Education: Artificial Intelligence	186
A02	Systematic literature review on opportunities, challenges, and future research recommendations of artificial intelligence in education	Thomas K.F. Chiu et al.	2023	Computers and Education: Artificial Intelligence	132
A03	Myths, mis- and preconceptions of artificial intelligence: A review of the literature	Arne Bewersdorff et al.	2023	Computers and Education: Artificial Intelligence	232

Cód.	Título	Autores	Año	Revista	Pal.
A04	Opportunities of artificial intelligence for supporting complex problem-solving: Findings from a scoping review	Srecko Joksimovic et al.	2023	Computers and Education: Artificial Intelligence	206
A05	Application and theory gaps during the rise of Artificial Intelligence in Education	Xieling Chen et al.	2020	Computers and Education: Artificial Intelligence	253
A06	A multi-perspective study on Artificial Intelligence in Education: grants, conferences, journals, software tools, institutions, and researchers	Xieling Chen et al.	2020	Computers and Education: Artificial Intelligence	172
A07	Human-centred learning analytics and AI in education: A systematic literature review	Riordan Alfredo et al.	2024	Computers and Education: Artificial Intelligence	222
A08	A systematic review of artificial intelligence techniques for collaborative learning over the past two decades	Seng Chee Tan et al.	2022	Computers and Education: Artificial Intelligence	210
A09	AI technologies for education: Recent research & future directions	Ke Zhang y Ayse Begum Aslan	2021	Computers and Education: Artificial Intelligence	160
A10	Educational applications of artificial intelligence in simulation-based learning: A systematic mapping review	Chih-Pu Dai y Fengfeng Ke	2022	Computers and Education: Artificial Intelligence	201
A11	A scoping review on how generative artificial intelligence transforms assessment in higher education	Qi Xia et al.	2024	International Journal of Educational Technology in Higher Education	183
A12	A meta systematic review of artificial intelligence in higher education: a call for increased ethics, collaboration, and rigour	Melissa Bond et al.	2024	International Journal of Educational Technology in Higher Education	257
A13	Artificial intelligence for education and its emphasis on assessment and adversity quotient: a review	Jyoti Mudkanna Gavhane y Reena Pagare	2024	Education + Training	244
A14	A comprehensive exploration of artificial intelligence competence frameworks for educators: A critical review	Tamar Mikeladze et al.	2024	European Journal of Education	154
A15	A Systematic Review of Generative AI for Teaching and Learning Practice	Bayode Ogunleye et al.	2024	Education Sciences	216
A16	Generative Artificial Intelligence in Tertiary Education: Assessment Redesign Principles and Considerations	Che Yee Lye y Lyndon Lim	2024	Education Sciences	152
A17	A Systematic Review of AI-Driven Educational Assessment in STEM Education	Fan Ouyang et al.	2023	Journal for STEM Education Research	200
A18	Transforming Education: A Comprehensive Review of Generative Artificial Intelligence in Educational Settings through Bibliometric and Content Analysis	Zied Bahroun et al.	2023	Sustainability	203
A19	What Is the Impact of ChatGPT on Education? A Rapid Review of the Literature	Chung Kwan Lo	2023	Education Sciences	202
A20	Systematic review of research on artificial intelligence applications in higher education – where are the educators?	Olaf Zawacki-Richter et al.	2019	International Journal of Educational Technology in Higher Education	216

B. Reproducción de los resultados

Los datos y resultados del informe se regeneran desde la raíz del proyecto con:

```
pip install -r requirements-dev.txt -r requirements-ai.txt
python -m bibliometria.extraction.build_corpus     # PDF -> corpus.json
python scripts/build_embeddings.py                 # GloVe (subconjunto) y SBERT
python -m bibliometria.string_matching.benchmark   # benchmark de R3
pytest                                             # verificación (177 pruebas)
python main.py                                     # consola: R1, R2 y R3
```

C. Primeras fusiones del agrupamiento con average linkage

Tabla 36: Primeras 8 fusiones de *average linkage* sobre el corpus completo (TF-IDF). C_t es el clúster creado en el paso t.

t	**Fusión**	**Distancia**	**Tamaño**
1	A08 ∪ A10 → C1	0.6818	2
2	A06 ∪ A09 → C2	0.6960	2
3	A11 ∪ A17 → C3	0.7099	2
4	A12 ∪ A20 → C4	0.7235	2
5	A02 ∪ A03 → C5	0.7251	2
6	C2 ∪ C5 → C6	0.7458	4
7	A05 ∪ C6 → C7	0.7654	5
8	A16 ∪ C3 → C8	0.7720	3

Las primeras fusiones unen pares con temas cercanos: A08–A10 (aprendizaje colaborativo y aprendizaje basado en simulación), A06–A09 (estudios panorámicos del campo AIEd), A11–A17 (evaluación con IA en educación superior y en STEM) y A12–A20 (revisiones sobre IA en educación superior).